PBench - Benchmarking Relational Provenance Systems

Pengyuan Li
Illinois Institute of Technology
USA
pli26@hawk.illinoistech.edu

Davood Rafiei
University of Alberta
Canada
drafiei@ualberta.ca

Boris Glavic
University of Illinois, Chicago
USA
bglavic@uic.edu

ABSTRACT

Data provenance is fundamental to establishing trust in data, debugging complex computations, explaining query outcomes, and supporting auditing and compliance. Over the past two decades, several relational provenance systems have been proposed, but comparing them remains difficult because they differ not only in runtime, but also in the provenance models they support, the SQL features they handle, the representation they produce, and the extent to which captured provenance can support downstream tasks. Existing evaluations typically focus on a subset of these dimensions, making it difficult to understand the practical trade-offs among competing designs. We present *PBench*, the first benchmark for relational provenance systems that evaluates these dimensions jointly. Using PBench, we compare four representative provenance systems—GProM, ProvSQL, SQLProv, and SmokedDuck. Our results reveal clear trade-offs between capture cost, storage overhead, provenance expressiveness, and downstream utility.

PVLDB Reference Format:
Pengyuan Li, Davood Rafiei, and Boris Glavic. PBench - Benchmarking Relational Provenance Systems. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/pli26/PBench_2027.git

1 INTRODUCTION

Relational data provenance models how the outputs of a query are derived from its inputs [14, 22, 27, 31]. Provenance has become a core component of modern data management, supporting tasks such as debugging query results, explaining analytical outcomes, maintaining derived data products, reproducing computations, and auditing data-intensive workflows. The database community has proposed a rich collection of provenance models, representations, and systems. Yet despite this substantial progress, the field is lacking a standardized benchmark that enables principled comparisons of relational provenance systems.

A key challenge in comparing provenance systems lies in the diversity of their underlying models and architectures. At the tuple level, systems may capture lineage [17, 19] , which records the input tuples contributing to each output tuple, or more expressive forms such as provenance polynomials [6, 55] that encode how input tuples are combined to derive a result. Systems also differ in how provenance is captured and stored, ranging from query rewriting approaches to database extensions and specialized execution engines. These choices affect SQL feature support, runtime overhead, storage requirements, and supported tasks.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

Example 1.1. *Consider the database shown in Fig. 1 and the following query Q_{emp} that returns employees from region "North".*

```
SELECT name, salary
FROM  Dept JOIN Emp ON (did = deptid)
WHERE region = 'North';
```

Fig. 2 shows the provenance of each result of Q_{emp} according to the lineage [19] and provenance polynomials [26] models. Lineage captures which input tuples were used to derive a result, e.g., Alice (r_1) is based on the Emp *tuple t_1 and the department tuple s_1. Provenance polynomials make explicit how input tuples are combined. In this example, r_1 is the result of joining t_1 with s_1, represented as multiplication.*

Provenance is used to support a variety of tasks, including tracing erroneous results back to their sources (backward tracing), identifying outputs affected by input errors (forward tracing), and explaining complex query behavior. Beyond these user-facing applications, provenance also enables probabilistic query processing [56, 59], hypothetical reasoning (e.g., what-if and how-to queries) [12, 16, 18, 42], reproducibility of computational experiments [15, 57], and efficient maintenance of derived data products under updates [32]. As Ex. 1.1 illustrates, different provenance models vary in expressiveness. These differences have important practical implications as applications differ in the minimal expressiveness required to support them: less expressive models are insufficient while too expressive provenance will potentially incur unnecessary overhead.

Despite the maturity of the field, relational provenance lacks a standardized benchmark. Existing systems are typically evaluated in isolation, often using workloads tailored to their particular provenance model, implementation strategy, or target application. Thus, results are often difficult to compare, and fundamental questions remain unanswered: How does provenance overhead vary across SQL features? When is a more expressive provenance representation worth its additional cost? Do compact representations remain advantageous once downstream tasks such as debugging or reproducibility are considered? Without a common benchmark, these trade-offs remain poorly understood.

Contributions. We introduce PBench, a benchmark for systematic evaluation of relational provenance systems. PBench evaluates systems along three complementary dimensions: (i) SQL-centric workloads that stress relational operators, query structures, and feature coverage; (ii) provenance-centric workloads that vary provenance size, complexity, and representation characteristics; and (iii) use-case-driven workloads that evaluate end-to-end tasks such as debugging, tracing, and reproducibility. By jointly considering these

eId	name	deptId	salary	$\mathbb{N}[X]$
1	Alice	101	110,000	t_1
2	Peter	101	110,000	t_2
3	Bob	102	80	t_3
4	John	103	95,000	t_4
5	George	104	155,000	t_5

Emp

dId	deptName	region	$\mathbb{N}[X]$
101	Sales	North	s_1
102	IT	North	s_2
103	Marketing	South	s_3
104	HR	East	s_4
105	Finance	North	s_5

Dept

Figure 1: Example employee database with relations Emp and Dept

	name	salary	Lineage	$\mathbb{N}[X]$
r_1	Alice	110,000	$\{t_1, s_1\}$	$t_1 \cdot s_1$
r_2	Peter	110,000	$\{t_2, s_1\}$	$t_2 \cdot s_1$
r_3	Bob	80	$\{t_3, s_2\}$	$t_3 \cdot s_2$

Figure 2: Result of Q_{emp} with lineage and provenance polynomials

	region	Witness	Lineage	Polynomials ($\mathbb{N}[X]$)
r_1	North	$\{t_1, s_1\}$	$\{t_1, t_2, s_1\}$	$\delta(t_1 \cdot s_1 + t_2 \cdot s_1)$
r_2	East	$\{t_5, s_4\}$	$\{t_5, s_4\}$	$\delta(t_5 \cdot s_4)$

Figure 3: Result and provenance for Q_{region}

dimensions, PBench enables evaluation beyond provenance-capture performance alone, exposing trade-offs among provenance expressiveness, runtime overhead, storage cost, and downstream usability. The benchmark aids practitioners in selecting the right provenance system for their workload as well as provides researchers with a tool to systematically evaluate provenance systems and algorithms. Using PBench, we conduct the first comprehensive experimental comparison of representative provenance systems, including SmokedDuck, ProvSQL, SQLProv, and GProM, on both PostgreSQL and DuckDB. Systems tightly integrated with the query execution engine achieve the lowest capture overhead, while systems optimized for provenance consumption can outperform lower-overhead alternatives when complete debugging or tracing workflows are considered. For certain query constructs such as nested subqueries, the ability to factorize provenance is critical for performance. We further show that highly expressive provenance models may incur substantial storage and runtime costs that are unnecessary for many applications, and that provenance-capture strategies, representation choices, and implementation choices often have a substantial impact on end-to-end efficiency. These findings suggest that provenance systems should be designed and evaluated with their target use cases in mind rather than optimized for a single performance metric. and provide practical guidance for the design of future provenance-aware database systems.

The remainder of this paper is organized as follows. We introduce common provenance models in Sec. 2, discuss motivating use cases for database provenance and identify their requirements in Sec. 3, and give an overview of the evaluated systems and their design decisions in Sec. 4. We present our benchmark PBench in Sec. 5 and use this benchmark to evaluate the systems in Sec. 7. Related work is discussed in Sec. 8. We present conclusions and discuss future work in Sec. 9.

2 PROVENANCE MODELS

A key challenge in evaluating provenance systems is that they support different provenance models [11, 14, 22, 31]. These models differ in at which granularity they track provenance (e.g., tuples or attribute values) and in whether they only track data dependencies (e.g., which input tuple does a query result tuple depend on) or also model how input tuples are combined to produce a result. These differences have important implications for the functionality, performance, and storage requirements of provenance systems. This section introduces the provenance models considered in this benchmark and highlights their trade-offs. We refer the reader to [14, 22, 31, 34] for detailed treatments.

2.1 Witnesses

The notion of a witness for a query result $t \in Q(D)$ was first introduced by Buneman et al. [11] to denote a subset $w \subseteq D$ of the input database D that is *sufficient* for producing t:

$$t \in Q(w)$$

As already observed in [11], a witness may contain redundant tuples that are not needed to produce the result. To avoid such redundancy, we can require witnesses to be *minimal*:

$$\nexists w' \subset w : t \in Q(w')$$

If the goal of provenance is to merely reproduce query results, then using a single witness as provenance is sufficient. In fact, in this case we can also use a witness for the complete query result as in [33] where $w \subseteq D$ is a witness if $Q(w) = Q(D)$. We will call such witnesses "holistic" to distinguish them from witnesses for individual query results. Determining minimal holistic witnesses is NP-complete in data complexity [33].

Example 2.1. *Consider query Q_{region} that returns regions which have departments with employees that earn more than $100k:*

```
SELECT DISTINCT region
FROM Dept d, Emp e
WHERE d.dId = e.deptId AND salary > 100,000;
```

On the database from Fig. 1 this query returns two results as shown in Fig. 3. Consider the first result tuple r_1. The witness $\{t_1, s_1\}$ shown on the right of the tuple is sufficient for producing r_1.

2.2 Lineage

When it is important to identify *all* inputs that contributed to a result as part of some derivation, a single witness is insufficient; a more expressive provenance model is needed. *Lineage* [1] captures the set of all input tuples that participate in at least one derivation of a result. This concept was first coined in [60] and later formalized based on sufficiency and necessity [19].[2] For instance, for the query from Ex. 2.1, the lineage of row r_1 is $\{t_1, t_2, s_1\}$ as there are two witnesses (alternative ways to derive r_1): $\{t_1, s_1\}$ and $\{t_2, s_1\}$.

[1] The term lineage is also used in the probabilistic database literature to refer to a different type of provenance, namely Boolean lineage formulas over input tuples. In the K-relational model, this corresponds to semiring $PosBool(X)$.

[2] For some approaches, Lineage is used in a looser sense and may not contain tuples from all possible derivations of a result under some operations, e.g., nested subqueries.

2.3 K-relations and Provenance Polynomials

Lineage identifies which tuples contributed to a result, but does not explain *how* they contributed. Provenance polynomials address this limitation by explicitly encoding the derivation structure of query results. Green et al. [26] introduced K-relations, a generalization of relations where tuples are annotated with elements from a commutative semiring K. A semiring consists of a set K and two binary operations (addition and multiplication) over K. For an appropriate choice of semiring, K-relations can model several provenance models including Lineage and MWhy-provenance [11] (the set of all minimal witnesses for a query). Furthermore, the semiring $\mathbb{N}[X]$ of *provenance polynomials*, whose elements are polynomials over variables X representing input tuples, subsumes all provenance models expressible in the K-relational model. Using semiring $\mathbb{N}[X]$, each input tuple is annotated with a distinct variable. These annotations are propagated during query evaluation such that the annotation of a result tuple t encodes both which input tuples contributed to it and how these tuples were combined. Addition represents alternative derivation (e.g., union), while multiplication represents joint use of tuples (e.g., join).

Example 2.2 (Provenance Polynomials). *Consider again query Q_{region} and its result shown in Fig. 3. The provenance polynomial for tuple r_1 = (North) encodes two alternative derivations: joining t_1 with s_1 or joining t_2 with s_1. Furthermore, the result was subjected to duplicate elimination (DISTINCT, represented as δ in the polynomial).*[3]

Provenance polynomials provide a strong guarantee: given the polynomial for a query result, we can determine how the result would change if the multiplicities of input tuple were modified. [4]

Example 2.3 (Maintaining Results). *Continuing with Ex. 2.2, consider the polynomial for tuple r_1:*

$$\delta(t_1 \cdot s_1 + t_2 \cdot s_1)$$

Let us assume that we deleted the input tuple t_1 (setting its multiplicity to zero) and increased the multiplicity of the input tuple s_1 to 2 (there are two copies in the input). Will r_1 =(North) still be in the result? To answer this question, we evaluate the polynomial (function δ returns 1 for any non-zero input) by substituting each variable with the multiplicity of the corresponding input tuple:

$$\delta(0 \cdot 2 + 1 \cdot 2) = \delta(0 + 2) = \delta(2) = 1$$

Sine the result is non-zero, tuple r_1 is still in the query result.

2.4 Coarse-grained Over-Approximations

For some applications, it is sufficient to capture provenance at a coarser granularity than individual rows, trading precision for smaller provenance size. For instance, Niu et al. [48] propose *provenance sketches*, which track provenance at the granularity of fragments induced by a horizontal partitioning of a relation. This is also related to summarization of provenance polynomials [2, 3].

Example 2.4. *For instance, consider a range-based horizontal partition of table* Dept *on attribute* region. *The coarse-grained provenance of Q_{region} using the partition would contain the two fragments $f_{North} = \{s_1, s_2, s_5\}$ and $f_{East} = \{s_4\}$ as only these fragments contain provenance. This is an over-approximation of lineage, because the tuple s_5 does not belong to the lineage of result tuple r_1 or r_2.*

[3]The δ operator is part of an extension that models duplicate elimination [5].
[4]Handling insertions of new tuples requires more expressive provenance models extending the K-relational framework. [28, 29, 37, 62]

3 USE CASES

The usefulness of a provenance system ultimately depends on the tasks it enables. This section reviews use cases for relational provenance and identifies their requirements in terms of provenance models and system functionality. One major insight is that evaluating provenance systems solely based on capture performance is insufficient: a provenance representation that is efficient to capture may incur additional overhead for downstream tasks.

3.1 Debugging Query Results

By tracking which outputs of a query depend on which inputs, database provenance is a useful tool for debugging. *Backward debugging* starts from an unexpected result and seeks the input tuples responsible for it. For instance, assume that in our running example we consider result tuple r_3 (result of Q_{emp}) suspicious, because its salary value is unreasonably low. The provenance of this tuple unveils that this is due to the low salary of t_3. *Forward debugging* starts from known problematic data and determines which query results may be affected. For instance, if some data is imported from an unreliable source, then we would like to understand which query results are affected by this data. In both cases of debugging, provenance narrows the search space by identifying relevant data dependencies.

Debugging requires provenance that captures all contributing tuples across all derivations such as polynomials or Lineage. Single witnesses are insufficient because they do not identify all problematic data dependencies, Note that we assume that the query itself is correct. Fixing issues with the query requires orthogonal techniques which are beyond the scope of this benchmark [4, 9, 10, 13, 20, 39, 40, 43, 58].

3.2 Reproducing Results

Reproducibility of computational experiments requires, at a minimum, that rerunning an experiment in a different environment should produce the same result. Provenance enables the creation of *"reproducibility packages"* [15, 57] which contain sufficient data and instructions to ensure this. For computations that involve a relational database, a reproducibility package should only include subsets of the database(s) accessed by the experiment that are sufficient for guaranteeing that all executed statements return the same result as in the original execution [50, 51]. Because a witness is by definition sufficient to regenerate a query result, witnesses are the least expressive provenance model required for reproducibility. More expressive models are overkill for this task.

Example 3.1. *Assume that query Q_{region} (Ex. 1.1) is part of a computational experiment. A reproducibility package for this experiment only needs to include a single holistic witness for this query. For Q_{region} such a witness can be generated by computing the union of a witness for each result tuple (Fig. 3):*

$$w = \{t_1, t_5, s_1, s_4\}$$

3.3 Probabilistic Databases

Probabilistic databases [59] model data uncertainty through a probability distribution over possible worlds. Evaluating queries over such databases requires computing the marginal probability for each possible result tuple to appear in the query's output. Provenance polynomials provide a symbolic representation of the derivations of a result tuple and can be used to compute marginal probabilities under bag semantics [56, 59]. Under set semantics, Boolean formulas over variables representing input tuples (semiring $PosBool(X)$) are sufficient. Probabilistic query evaluation requires knowing the exact derivation structure of query results. Consequently, witnesses and lineage are insufficient.

Example 3.2 (Probabilistic Databases). *Consider the polynomial for result tuple r_1 of query Q_{region}. By substituting addition with "or" and multiplication with "and", we get the "Lineage" formula:*

$$(t_1 \wedge s_1) \vee (t_2 \wedge s_1) = s_1 \wedge (t_1 \vee t_2)$$

Assuming input tuple probabilities $p(t_1) = 0.8$, $p(t_2) = 0.9$, $p(s_1) = 0.5$, the marginal probability of r_1 is the same as the probability of its Lineage formula evaluating to true [59]:

$$p(r_1) = p(s_1)\cdot(1-((1-p(t_1))\cdot(1-p(t_2)))) = 0.5\cdot(1-0.2\cdot0.1) = 0.49$$

3.4 View Maintenance & View Update

View maintenance and view-update problems require reasoning about how changes to input data affect query results. In view maintenance [30] and what-if analysis [7, 45] the output of a query should be updated to reflect an update to the database. For any deletion of a subset of inputs, the provenance polynomial of a query result can be used to evaluate the impact of this deletion by replacing the variables for deleted tuples with zero and reevaluating the polynomials. [5] In view update (*How-to* analysis [42]), a desired change to the output of a view should be translated into an update to the database such that running the view's query over the updated database yields the requested update to the view's result. If we limit updates to deletions, then this is referred to as *delete propagation* [12, 35]. Using provenance polynomials, this boils down to constraint-solving [41, 42]: find an assignment of input variables that determines which tuples to delete. Lineage and, thus, also a single witness, are insufficient for view maintenance and update, as it does not track how input tuples were combined, necessary for determining how deletions affect results.

Example 3.3 (What-If and How-to). *Suppose Alice does not work for the company anymore (tuple t_1 is deleted). To determine the impact of this update on Q_{region}, we can replace t_1 with 0 in the polynomial for every result tuple. Any tuples whose polynomial now evaluates to zero will no longer be in the result, e.g., for r_1:*

$$\delta(0 \cdot s_1 + t_2 \cdot s_1) = \delta(t_2 \cdot s_1)$$

Thus, r_1 is still in the result. Now consider a How-to query. To delete result tuple r_1 but keep r_2, we have to solve the following constraints:

$$t_1 \cdot s_1 + t_2 \cdot s_1 = 0 \qquad t_5 \cdot s_4 > 0$$

For example, one possible solution is to delete s_1 (setting $s_1 = 0$).

[5] With the extensions from [5], aggregate queries are also supported. Furthermore, there are provenance models that also deal with negation and can support insertions [28, 38]

3.5 Optimizing Data Management

Provenance also enables new optimizations in relational databases. Since the provenance of a query is sufficient to reproduce its result, it acts as a compact summary of the data relevant to that query. For example, the provenance of a query Q_1 can be materialized and reused to accelerate the evaluation of a related query Q_2 by executing Q_2 over Q_1's provenance rather than the full database [48, 61]. Provenance can also guide physical design decisions: data that frequently appears in query provenance can be prioritized for placement in faster storage or caches [24]. These applications typically only require identifying data that may influence a result, not how it contributes to the result. Consequently, lightweight provenance models such as holistic witnesses or coarse-grained approximations thereof (e.g., provenance sketches [48]) are typically sufficient.

Example 3.4 (Optimization). *Assume we have captured coarse-grained provenance for Q_{region} as explained in Ex. 2.4 containing the fragments f_{North} and f_{East} for regions "North" and "East". The following query differs from Q_{region} only in using a stricter* `WHERE` *condition. This query can be evaluated on $\{f_{North}, f_{East}\}$ as its provenance is contained in Q_{region}'s provenance.*

```
SELECT DISTINCT region
FROM Dept d, Emp e
WHERE d.dId = e.deptId
and salary > 115,000;
```

4 SYSTEM OVERVIEW

For the benchmark evaluation we only consider reasonably complete systems. We selected a representative set of systems regarding the architectural / algorithmic decisions. Table 1 compares these systems GProM [6], ProvSQL [55], SmokedDuck [44] and SQLProv [46] on their system architecture & how provenance is captured, supported provenance models, supported SQL features, and provenance representation and query capabilities. We show how these systems capture provenance for an example query in Sec. B.

4.1 GProM

GProM [6] compiles provenance requests into pure SQL, supporting PostgreSQL, SQLite, DuckDB, MonetDB, and Oracle as backends. For positive relational algebra, GProM computes a flat-relational encoding of Provenance Polynomials while for more general queries its model encodes at least Lineage. In addition, GProM supports coarse-grained provenance. GProM uses a propagation approach: the user selects which attributes of input tables to treat as provenance and GProM instruments the input query Q to construct a SQL query Q_{rewr} that propagates these annotations such that each result tuple of Q_{rewr} pairs a result tuple of Q with the selected attributes of tuples from its provenance. GProM exposes provenance capture as a new SQL query feature and, thus, full SQL can be used to combine capture and querying of provenance in a single query.

4.2 ProvSQL

ProvSQL [55] is a PostgreSQL extension that supports provenance polynomials encoded as circuits. Each circuit node is identified through an automatically generated UUID. To enable provenance tracking, users invoke the UDF `add_provenance`, which augments each table with a new ProvSQL column associating each row with a variable (encoded as a UUID). During provenance capture, queries are instrumented with UDF calls that create circuits. Circuit nodes

Category	Property	ProvSQL	GProM	SmokedDuck	SQLProv
Architecture	Type	Extension	Middleware	Modified Execution Engine	Extension
	Backend	PostgreSQL	Multiple	DuckDB	PostgreSQL
Capture	Workflow	Single-phase	Single-phase	Two-phase	Two-phase
	Strategy	Circuit propagation	Annotation propagation	I. Mapping + II. Backtrace	I. Mapping + II. Replay
Provenance	Lineage	✓ (via polynomials)	✓	✓	✓
	Witnesses	✓ (via polynomials)	✓ (via lineage)	✓ (via lineage)	✓ (via lineage)
	Provenance Polynomials	✓	✓(SPJ)	✓	✗
	Coarse-grained	✗	✓	✗	✗
SQL Support	SPJ (+DISTINCT)	✓	✓	✓	✓
	Aggregation	Partial	✓	✓	✓
	Set Operations	Set only	✓	✓	✗
	Nested Queries	✗	✓	✓	✓
	Additional Features	–	Top-K	Top-K	Window, Recursive, Top-K
Storage	Additional Storage	Circuits in mmap'ed files	None	In-memory Maps	Logging Tables

Table 1: Comparison of evaluated provenance systems. "via X" means that this type of provenance requires first generating provenance of type X and then translating it into the desired type of provenance.

and edges are created by sending IPC messages to a background worker, which stores the circuits in memory-mapped files. Each result tuple of the instrumented query contains an additional `provsql` attribute that stores the UUID of the root node of its polynomial circuit. ProvSQL provides UDFs for constructing a string representation of a circuit and for evaluating it using a user-provided *"provenance mapping"* that assigns semiring elements to variables.

4.3 SmokedDuck

SmokedDuck [44] extends the execution engine of DuckDB with native provenance support. Capture is performed in two phases. During query execution, instrumented physical operators record in-memory tables that map each output tuple to the input tuple identifiers that contributed to it. Provenance is reconstructed on demand for a single or all result tuples of a query by recursively joining these mapping tables to trace dependencies back to the base tables. This top-down reconstruction during the 2nd phase avoids materializing provenance for intermediate results that do not eventually contribute to the final query result. SmokedDuck also supports provenance polynomials.

4.4 SQLProv

SQLProv [46] captures fine-grained provenance for SQL through query instrumentation. Implemented on top of PostgreSQL, the approach supports both tuple- and attribute-level provenance. Provenance is captured in two phases. In the first phase, the input query is instrumented to create tuple identifiers and log input-output mapping for each operator, similar to SmokedDuck. These mappings are recorded in logging tables through injected UDF calls. In the second phase, the input query is instrumented to operate on provenance identifiers instead of data values. Additional UDFs retrieve the logged mappings, while computations on values are replaced with set-union operations over provenance identifiers. The resulting tuples (or attribute values) store sets of input tuple (or attribute value) identifiers that constitute their Lineage.

4.5 System Comparison

The evaluated systems differ along three primary dimensions: provenance-capture strategy, provenance representation, and storage architecture. GProM and ProvSQL construct provenance during query execution and expose it immediately to the user, whereas SQLProv and SmokedDuck use a two-phase workflow in which data dependencies for parts of the query are first recorded and provenance is reconstructed later. This design can be advantageous when provenance is captured once but queried multiple times, as the capture cost is incurred only once. The systems also differ in how provenance is represented. GProM encodes provenance as a mapping table that associates each result tuple with contributing tuples, potentially duplicating the result tuple if necessary. This "normalized" representation can lead to redundancy when tuples from different inputs independently contribute to a result, e.g., for queries with multiple nested subqueries. For each result tuple, ProvSQL uses an additional attribute to store the UUID of the root gate of a circuit for the row's provenance polynomial, the circuits themselves are stored in memory-mapped files. SQLProv represents provenance of a result tuple as a set of tuple ids encoded as a Postgres array value. SmokedDuck displays the row ids of provenance tuples. Finally, the systems differ in their storage requirements. GProM captures provenance by running rewritten queries and requires no extra storage, whereas extra storage is needed for ProvSQL (the mmap files storing circuits), SQLProv (the logging tables) and SmokedDuck (in memory mapping tables).

5 PBENCH

We now introduce PBench, our benchmark for evaluating relational provenance systems. PBench evaluates the entire provenance lifecycle, including capture, storage, querying, and downstream use. The benchmark consists of three complementary workload families that stress different aspects of provenance systems: (i) application-oriented tasks derived from common provenance use cases, (ii) workloads that vary provenance size and structural properties, and (iii) workloads that exercise increasingly complex SQL constructs. Together, these workloads expose trade-offs among provenance expressiveness, capture overhead, storage requirements, and downstream usability. We first discuss the benchmark design, evaluation metrics, and the data generation process before presenting the individual benchmark tasks. As in TPC-H and other benchmarks, a *scale factor* P_{SF} controls the size of the input database and can be used to evaluate scalability of systems in data size.

Notation	Description
P_{SF}	Scale factor for the benchmark
$P_{\%\|Q(D)\|}$	Fraction of query result tuples for which provenance is requested
$P_{\%\|D\|}$	Fraction of input tuples which should be traced to a query's result
P_{tot}	Total provenance size for all result tuples of a query
P_{res}	Average size of the provenance of a single result tuple
P_{fact}	Ratio between the size of the factorized provenance polynomial and the size of its equivalent sum-of-product (SOP) representation
$P_{wit/lin}$	Ratio between the size of a single witness and the size of the full lineage of a query
T_{e2e}	End-to-end task runtime (provenance capture + querying + additional task-specific steps)
T_{cap}	Runtime for capturing provenance
T_{query}	Runtime of querying provenance data
T_{task}	Runtime of a task that uses queried provenance
T_{prep}	Runtime for preparing a database to be able to capture provenance
S_{cap}	Storage overhead incurred by a system to run a task
S_{db}	Storage overhead for preparing a database for provenance capture

Figure 4: Parameters and metrics of the benchmark

5.1 Design Considerations

PBench organizes benchmark tasks into three workload families.

Use-case-driven workloads. These workloads evaluate complete provenance workflows derived from practical applications such as debugging, reproducibility, and probabilistic query processing. They measure not only provenance capture but also the cost of consuming provenance in a task-specific manner.

Provenance-centric workloads. These workloads isolate the effect of provenance characteristics such as total provenance size, provenance size per result, intermediate provenance volume relative to final provenance size, and factorization. They reveal how provenance representation affects scalability.

Query-centric workloads. These workloads evaluate how query structure and SQL features influence provenance capture and storage. They stress systems with increasingly complex relational operators and language constructs.

Fig. 4 shows notation we use for parameters (that are varied in individual tasks) and metrics used in PBench.

5.2 Metrics

For each task, PBench requires reporting *runtime* and *storage overhead*. Depending on the workload, end-to-end time T_{e2e} is decomposed into capture (T_{cap}), provenance querying (T_{query}), and task execution (T_{task}):

$$T_{e2e} = T_{cap} + T_{query} + T_{task} \quad \text{(Task execution time breakdown)}$$

For systems that separate provenance capture into logging and reconstruction phases (e.g., SmokedDuck and SQLProv), capture time is further decomposed into T_{log} and T_{trace}. Storage overhead (S_{cap}) includes all additional data generated for provenance tracking, whether written to disk (e.g., ProvSQL stores provenance circuits as external files) or stored as database tables (e.g., SQLProv logs provenance in relational tables). Furthermore, some systems require augmenting the input database to enable provenance tracking. For these systems, we also report the one time storage overhead of augmenting the base data (S_{db}) and the time (T_{prep}) needed to prepare the input database.

5.3 Data Generation

PBench combines standard decision-support data with synthetic datasets specifically designed to control provenance characteristics.

Table	#Col.	#Rows
R^n_{fp}	8	$n \cdot P_{SF}$
R^c_{join}	4	$5 \cdot P_{SF} \cdot 10^4$
R^s_{join}	4	$25 \cdot P_{SF} \cdot 10^3$
R^1_{join}, R^2_{join}	5	$P_{SF} \cdot 10^4$

Table	#Col.	#Rows
$R^n_{\#grp}$	4	$n \cdot P_{SF} \cdot 10^3$
$R^n_{grpsize}$	4	$n \cdot P_{SF} \cdot 10^3$
R^1_{fac}, R^2_{fac}	5	$P_{SF} \cdot 10^4$
R_{graph}	2	$P_{SF} \cdot 10^5$

Table 2: Synthetic tables used in the benchmark

We use the *TPC-H* dataset scaled by the benchmark scale factor P_{SF}, together with 17 synthetically generated tables that allow us to independently vary provenance size, provenance factorization, query selectivity, and query structure. Table 2 summarizes the characteristics of all synthetic tables used in the benchmark. All attributes in synthetic tables are uniformly distributed. Table R^n_{fp}, for $n \in \{1k \cdot P_{SF}, 100k \cdot P_{SF}, 2M \cdot P_{SF}, 10M \cdot P_{SF}\}$, varies the number of distinct attribute values, allowing control over the provenance size per output tuple. Tables R^c_{join}, R^s_{join}, R^j_{join} and R^{jj}_{join} contain attributes with at most $1K \cdot P_{SF}$ distinct values and are used in join conditions to control join selectivity and result cardinality. Tables $R^n_{\#grp}$ and $R^n_{grpsize}$ are designed for aggregation queries, where n controls the number of groups and the number of tuples per group, respectively. Tables R^1_{fac} and R^2_{fac} are used to control the degree of provenance factorization. Finally, R_{graph} represents an edge relation of a graph with $1K \cdot P_{SF}$ nodes and $10^5 \cdot P_{SF}$ edges, and is used to evaluate recursive queries such as reachability. We show the SQL code for every benchmark query and provide additional details about these queries such as intermediate result sizes in Sec. A.

5.4 Use-case-driven Workloads

Use-case-driven workloads evaluate complete provenance workflows rather than provenance capture alone. In particular, tasks that require full input or output rows may impose additional work on systems that return only tuple identifiers, since these identifiers must be joined back with the base tables or query result to retrieve attribute values. Unless stated otherwise, we decompose the end-to-end execution time as shown above into: (1) capture provenance, (2) query captured provenance to retrieve relevant rows when needed, and (3) perform the task-specific computation over the retrieved data. Systems may combine these steps unless explicitly disallowed. We present additional details about the system-specific implementations in Sec. B.1.

T1: Backward Debugging traces a fraction $P_{\%|Q(D)|} \in \{1\%, 10\%, 50\%\}$ of erroneous result tuples back to the input tuples responsible for producing them. The number of provenance tuples per $P_{\%|Q(D)|} = 1\%$ is $\sim 10K \cdot P_{SF}$. We evaluate backward debugging using a query Q_{BDebug}, a join followed by group-by aggregation that produces 1K result tuples. This workload evaluates whether a system can efficiently compute provenance for a selected subset of query results. The workflow follows the standard protocol, where the last step filters provenance to return explicit data for the selected result tuples. Systems may push the selection into the capture phase. We evaluate backward debugging using query Q_{BDebug}, a join followed by group-by aggregation that produces 1K result tuples.

T2: Forward debug starts from a fraction $P_{\%|D|} \in [1\%, 5\%]$ of problematic input tuples and identifies query results that may be affected

by them. This workload evaluates the ability of a provenance system return all result rows whose provenance contains at least one erroneous input tuple. We use query templates: Q_{FD1}, a join followed by aggregation, and Q_{FD2}, a join query. See Sec. A.1.2. We use table $R^{1k}_{\#grp}$. Additional forward debugging variants are described in Sec. C.1.1, including determining which queries in a workload are affected by erroneous inputs and mapping each erroneous input to the result tuples it affects.

T3: Reprod. evaluates whether a system can compute a subset of the input data sufficient to reproduce a query result. The workflow therefore consists of (i) capturing and materializing provenance ($T_{cap} + T_{query}$), and (ii) re-executing the query on the materialized provenance (T_{task}). We separate these steps to reflect the creation and subsequent use of a reproducibility package. We use two workloads: query Q_{Lin} is a join followed by a group-by `min` aggregation. This query joins R^{10M}_{fp} with R^{c}_{join}. The total size of a holistic witness (which is equal to the size of the lineage) for this query is $\sim 10M \cdot P_{SF}$ $((10M + 10K) \cdot P_{SF})$ tuples. The other is a query template Q_{Wit}, a group-by aggregation on top of a join. We vary join selectivity through parameter $n \in \{1k \cdot P_{SF}, 100k \cdot P_{SF}, 2M \cdot P_{SF}\}$ which affects the left input of the join (table R^{n}_{fp}), resulting in witness sizes of $P_{wit/lin} \in \{0.01\%, 1\%, 20\%\}$ relative to the full lineage ($\sim 10M \cdot P_{SF}$).

T4: What-If evaluates the ability of systems to incrementally maintain the result a query using materialized provenance under hypothetical deletions of input tuples. We consider the query template Q_{What}, a SPJ query, and investigate a scenario in which a subset $r \in \{1\%, 10\%, 50\%\}$ of input tuples in the provenance is deleted ($\{10^4 \cdot P_{SF}, 10^5 \cdot P_{SF}, 5 \cdot 10^5 \cdot P_{SF}\}$). We measure the capture time (T_{cap}) and cost of computing a delta for the query result based on the materialized provenance and a delta for the input (T_{query}).

5.5 Provenance-centric Workloads

These workloads isolate the impact of provenance characteristics on system performance. For two-phase systems such as SQLProv and SmokedDuck, the over-all runtime and the runtime of the first phase (T_{log}) should be reported.

T5: Prov. per Result fixes total lineage size $P_{tot} = 10M \cdot P_{SF}$ and varies average provenance per result P_{res} from 10 to 10^6. Query Q_{FPAgg} computes a group-by average query (table R^{10M}_{fp}) and computes averages; Q_{FPDist} computes the `DISTINCT` values of a grouping attribute.

T6: Ttl. Prov. Size varies P_{tot} from $100K \cdot P_{SF}$ to $10M \cdot P_{SF}$. Query Q_{VPGN} is a group-by aggregation. We fix P_{res}, the provenance size per result while vary the number of result tuples. Query Q_{VPGS} fixes the number of results and varies P_{res}.

T7: Interm. Prov. Size evaluates how systems deal with large provenance for intermediate results when the provenance of the final query result is small. Many provenance systems generate substantially more provenance for intermediate query results than is ultimately needed for the final query output. When only a small fraction of this provenance contributes to the final result, top-down construction (as in SmokedDuck) can avoid reconstructing unnecessary provenance, whereas bottom-up approaches (as in SQLProv) and single-phase bottom-up capture (as in GProM) cannot. We use query Q_{TopK} which is a top-k query over a sorted group-by result. We vary $P_{surviving}$, the fraction of intermediate results provenance that contributes to the final query result from $\frac{1}{2 \cdot P_{SF}}$ to $\frac{1}{1{,}000 \cdot P_{SF}}$ by varying k. The distinct values of $P_{surviving}$ we consider are $\{\frac{1}{2 \cdot P_{SF}}, \frac{1}{10 \cdot P_{SF}}, \frac{1}{100 \cdot P_{SF}}, \frac{1}{1{,}000 \cdot P_{SF}}\}$.

Several use cases, including probabilistic query processing and hypothetical reasoning (e.g., what-if and how-to queries), require provenance polynomials because they depend on the derivation structure of query results. While provenance capture provides the foundation for these use cases, they require additional functionality beyond basic capture and querying, which is only supported by some systems (e.g., FADE [45] built on SmokedDuck for what-if queries, and ProvSQL for probabilistic query evaluation). We therefore focus in this workload on the common prerequisite: capturing provenance polynomials. **T8: Provenance Polynomials** computes provenance polynomials for SPJ queries $Q_{join[1]}$, $Q_{join[2]}$, and $Q_{join[3]}$, which vary the number of joins, and query $Q_{joinagg}$, which is a group-by aggregation over a join. We measure capture time (T_{cap} with the breakdown into T_{log} and T_{trace} for 2-phase systems) and storage overhead (S_{cap}). For SPJ queries, the size of generated polynomials is constant in the data size and linear in the number of joins. For $Q_{joinagg}$, each polynomial contains $\sim 10K \cdot P_{SF}$ operations (gates).

T9: Factorization evaluates a system's ability to exploit factorization of provenance which is critical when there is significant redundancy in enumerating all combinations of input tuples that contribute to a result. Important examples include queries with multiple nested subqueries or aggregation / `DISTINCT` before joins. Note that this issues is not limited provenance polynomials, but also arises for lineage for systems like GProM that use flat representations. We compare runtime on pairs of queries which share the same provenance, but produce different factorization structure: joining before duplicate elimination (Q_{JD}) with duplicate elimination before joining (Q_{DJ}), and joining before aggregation (Q_{JA}) with preaggregation before join (Q_{AJ}). Factorization potential P_{fact} is measured as the size of factorized versus flat provenance. For provenance polynomials P_{fact} is computed as shown below

$$P_{fact} = \frac{\#p}{\#SOP(p)},$$

where $\#p$ is the size of a factorized polynomial for a query counted as the number of variable occurrences and $SOP(p)$ is the same measure for the corresponding unfactorized sum-of-products representation of the polynomial. We allow shared subexpressions (circuits). Smaller ratios mean more compression. For the two DISTINCT settings, ratios are about $1/(200P_{SF})$ and $1/(20P_{SF})$; for preaggregation the ratio is about $1/10$.

Note that for factorized provenance polynomials, we consider DAGs (circuits) rather than expression trees, i.e., identical subexpressions do not have to be repeated. Query Q_{JD} produces a flat SOP polynomial, whereas Q_{DJ} pushes duplicate elimination below the join to produce a factorized polynomial of the form $\delta(\delta(\sum_i r_i) \cdot \delta(\sum_j s_j))$ where $\sum_i r_i$ is from the left input of the join and $\sum_j s_j$ is from the right input. We consider two variants with different number of distinct values by using different attributes to project on, yielding $P_{fact} \approx \frac{1}{200 \cdot P_{SF}}$ and $P_{fact} = \frac{1}{20 \cdot P_{SF}}$ for Q_{DJ}. We also consider factorization with aggregation using queries Q_{JA} and Q_{AJ}. Q_{AJ} with $P_{fact} =\sim \frac{1}{10}$ pre-aggregates the left input before the join, yielding a factorization structure of $\delta(\sum_j \delta(\sum r_i) \cdot s_j)$.

5.6 Query-centric Workloads

These workloads evaluate how the structure of SQL queries and the language features they utilize affect the performance of provenance capture (T_{cap}) and storage overhead (S_{cap}). Unlike the use-case-driven workloads, they focus solely on provenance capture rather than downstream provenance consumption. As for provenance-centric workloads, for two-phase systems we report total execution time and the cost of the first phase.

T10: TPC-H measures provenance capture for all 22 queries of the TPC-H benchmark for scale factor equal to P_{SF}. We include TPC-H because it is a widely used decision-support benchmark containing diverse SQL constructs, including complex joins, aggregations, and nested subqueries.

T11: Sub-Q evaluates provenance capture for queries containing `WHERE` subqueries. It is required for systems to record provenance for tables accessed inside a nested subquery as otherwise the provenance may not even be sufficient for reproducing results. Different systems adopt different strategies for handling subqueries. For example, GProM rewrites them into lateral joins, whereas SQLProv does not capture provenance for tables appearing exclusively inside `WHERE` subqueries. We use query template Q_{WSub}, a group-by aggregation with `EXISTS` subqueries in the `WHERE` clause (see Sec. A.3.1), and vary the number of subqueries from one to three. The capability to factorize provenance is critical here, as the size of unfactorized provenance representations as used by GProM grows exponentially in the number of nested subqueries. **T12: Additional SQL language features** covers additional SQL language features such as argmin, window functions, recursive queries, and set operations. This task is described in Sec. C.2.

6 BENCHMARK COMPLIANCE

Table 3 shows for each system under consideration for which PBench tasks it can generate compliant outputs. When a system is not compliant, we indicate the cause of the failure. P: the system does not support the required provenance model; S: the query structure / SQL feature is unsupported; I: the system produces incomplete provenance. Running out of resources (time / memory / disk space) in the experimental evaluation is machine dependent and is not considered non-compliance. The listed provenance model is the least expressive model required for the task. Systems are allowed to use more expressive models as long as the required type of output is produced. Correctness of generated provenance is validated by (i) evaluating the query over the captured provenance to test that it produces the correct result and by comparing provenance across systems to detect potential non-compliance. Each detected compliance failure was then manually verified. GProM does not support provenance tracking for recursive queries and window functions. SQLProv does not support provenance polynomials, misses provenance dependencies in nested subqueries, and does not support capture for set operations. ProvSQL does not support capture for order-based operations (top-k), nested subqueries, window functions, recursive queries, and joins or additional operations on top of an aggregation result. SmokedDuck produces incomplete provenance for queries with `DISTINCT` over joins and does not support capture for window functions and recursive queries.

7 EXPERIMENTAL EVALUATION

We use PBench $P_{\text{SF}} = 1$ and $P_{\text{SF}} = 10$ to compare four representative relational provenance systems: SmokedDuck [44], ProvSQL [55], SQLProv [46], and GProM [6]. Among these, ProvSQL, SQLProv, and GProM run on PostgreSQL, while GProM and SmokedDuck support DuckDB. The evaluated systems differ substantially in their provenance models, capture strategies, and provenance representations, allowing us to study the trade-offs exposed by PBench. For systems that split capture into two phases, namely SQLProv and SmokedDuck, we report the cost of the first phase (T_{log}) and the cost of reconstructing provenance in the second phase (T_{trace}). Unless stated otherwise, blue percentages above bars in plots indicate the fraction of total capture time spent in phase one. For tasks that require returning full rows, systems that represent provenance as row ids must additionally join these identifiers with the base tables or query results to retrieve attribute values; GProM can avoid this step by directly producing provenance as relational data. All experiments were run on a machine with an Apple M2 Max CPU (12 cores) with 64GB of RAM, running macOS with Darwin Kernel 23.6.0. We use PostgreSQL 16.3 and DuckDB 1.0.0. We use DuckDB 1.0.0 because this is the version on which SmokedDuck is based. For evaluation on PostgreSQL, we use its default parallel settings and configure DuckDB to use a single thread for our experiment as SmokedDuck does not support parallel execution. Due to the fact that only one system (GProM) supports both PostgreSQL and DuckDB and that these two systems differ in many aspects beyond their storage model, comparing results across these databases is potentially problematic. For a meaningful comparison between GProM and SmokedDuck we did evaluate both systems on DuckDB 1.0.0 supported by SmokedDuck and without parallel execution. However, note that GProM does support parallel execution and performs better on newer DuckDB versions that feature a more powerful optimizer. Thus, its runtime should be treated as a lower bound. Experiments are repeated at least 10 times. We report median runtimes. For most experiments (97%) the relative variance (median-based) is below 1% (98.5% below 5%, and 99.8% below 10%). ProvSQL exhibited higher variance than the other systems, but even for the highest variance this did not impact the outcome because of the large runtime difference between systems.

7.1 Preparing the Database For Capture

We first evaluate the one-time cost of preparing the input database for provenance capture. This step is required only by ProvSQL and SQLProv. ProvSQL augments each input table with provenance tokens that map tuples to variables used in provenance polynomials. SQLProv creates provenance-specific versions of each table whose values are identifiers propagated during the second phase of provenance computation. For the synthetic PBench tables at $P_{\text{SF}} = 1$, the original database size is roughly 3.2GB (31.8 GB for $P_{\text{SF}} = 10$). Table 4 shows the runtime and storage overhead of preparing tables for provenance capture for the synthetic and *TPC-H* datasets at $P_{\text{SF}} = 1$ and $P_{\text{SF}} = 10$. Systems that have to modify the input database to support provenance introduce substantial one-time preparation and storage overhead.

Task / configuration	Required output and provenance model	GProM	SQLProv	ProvSQL	SmokedDuck
T1: Backward debugging	All input rows for selected outputs (*lineage*)	✓	✓	✓	✓
T2: Forward debugging	All outputs depending on erroneous inputs (*lineage*)	✓	✓	✓	✓
T3: Reproducibility	Sufficient input rows (*witness*)	✓	✓	✓	✓
T4: What-if deletion	Output membership after deletions (*polynomials*)	✓	P	✓	✓
T5: Per-result size (agg./dist.)	*lineage*	✓	✓	✓	✓
T6: Total size	*lineage*	✓	✓	✓	✓
T7: Intermediate size / top-k	*lineage*	✓	✓	S	✓
T8: Polynomial construction	*polynomials*	✓	P	✓	✓
T9: Factorization (agg./dist.)	*lineage*	✓	✓	✓	✓/ I
T10: TPC-H	*lineage* (incl. tracing into nested subqueries)	✓	✓/ I	✓/ S	✓
T11: Nested subqueries	*lineage* (incl. tracing into nested subqueries)	✓	I	S	✓
T12 : Multi-agg / argmin / recurs. / wind. / set op.	*lineage*	✓/ ✓/ S/ S/ ✓	✓/ ✓/ ✓/ ✓/ S	S/ S/ S/ S/ ✓	✓/ ✓/ S/ S/ ✓

Table 3: Task output and provenance model requirements and compliance of the evaluated systems.

Dataset	System	SF1		SF10	
		Time	Storage	Time	Storage
TPCH	SQLProv	39 s	5.1 GB	388 s	49.2 GB
	ProvSQL	37 s	0.4 GB	795 s	20.3 GB
Synthetic	SQLProv	105 s	10.1 GB	939 s	85.2 GB
	ProvSQL	122 s	2.2 GB	5051 s	49.4 GB

Table 4: Time and storage cost of constructing provenance related data for SQLProv and ProvSQL at SF1 and SF10.

	SF1			SF10		
	0.01%	1%	20%	0.01%	1%	20%
GProM	4.23 (0.06)	4.29 (6.00)	4.90 (115)	154 (0.6)	155 (58)	161 (1146)
SmokedDuck	0.21 (0.25)	0.22 (0.75)	0.26 (11)	2.44 (0.25)	2.59 (6.25)	2.69 (119)
SQLProv	4.26 (0.67)	4.66 (38.0)	11.14 (749)	154 (4.7)	158 (377)	221 (7475)
ProvSQL	4.26 (0.24)	4.59 (17.0)	9.05 (334)	154 (1.7)	156 (167)	192 (3338)

Table 5: Cost of constructing different sizes of witnesses at SF1 and SF10. We report execution time in seconds, followed by storage consumption in MB (in parentheses).

7.2 Use-case-driven Workloads

T1: Backward Debugging. Fig. 5a reports end-to-end runtime for backward debugging, where provenance is retrieved for a fraction $P_{\%|Q(D)|}$ of query results. GProM is the only evaluated system that can both restrict provenance capture to selected output tuples and directly return provenance as relational rows[6]. As a result, its runtime scales with $P_{\%|Q(D)|}$. In contrast, the other systems first capture provenance for all results and then retrieve the relevant input rows using row identifiers. The breakdown in Fig. 5b shows that capture time dominates their runtime. While SmokedDuck has the lowest first-phase capture time, the cost of reconstructing provenance in the second phase outweighs this advantage.

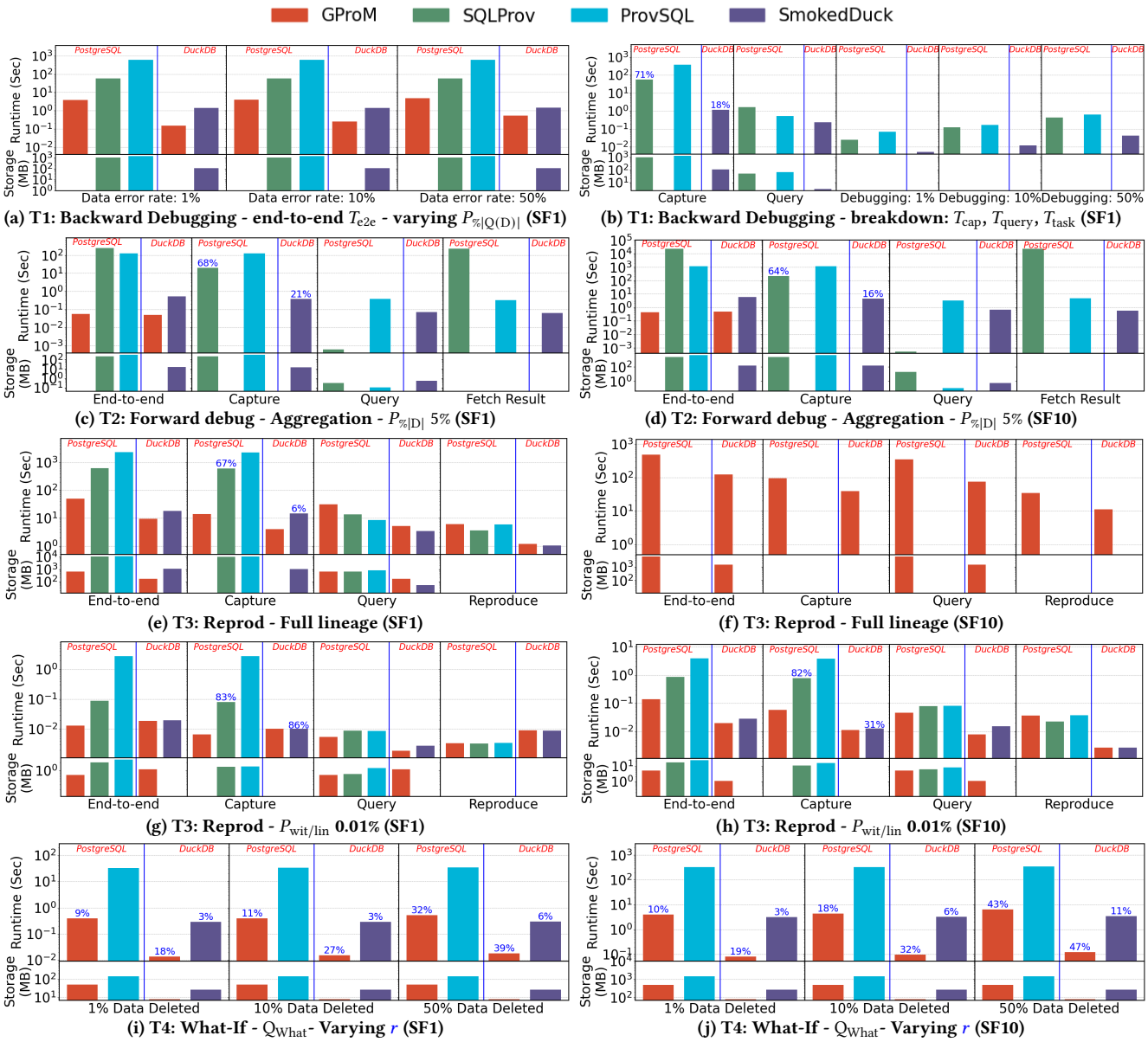
T2: Forward debug. Fig. 5c and Fig. 13b show the cost of identifying result tuples affected by erroneous input rows ($P_{\%|D|} = 5\%$) for aggregation and join queries (SF1 and SF10). For the aggregation query (Fig. 5c), SQLProv has lower capture cost than ProvSQL, but higher end-to-end runtime because affected rows must be identified by scanning provenance arrays for erroneous tuple identifiers. This is also why the performance gap between ProvSQL and SQLProv widens for SF10. Furthermore, this shows that efficient capture is not sufficient if the resulting representation is expensive to query for downstream tasks.

T3: Reprod. Fig. 5e to 5h compare reproducibility using lineage (overkill) and using a single witness at $P_{SF} = 1$ and $P_{SF} = 10$. For SF10 (full lineage) all systems but GProM time out. Since none of the evaluated systems can directly compute a single witness, for these systems using a witness is a trade-off: we have to capture lineage (provenance polynomials for ProvSQL) and extracting a witness from the lineage. Then this witness is used when reproducing querying results, lowering the cost of this step. Table 5 shows the time and storage costs of constructing a witness representing 0.01%, 1% and 20% of the full lineage. Nonetheless, this experiment demonstrates that witness-based reproducibility can provide significant runtime and storage savings when a query has many alternative derivations.

T4: What-If. For this task each system is given a delta deleting some input tuples represented as a table mapping variables in the provenance polynomials of query results to their new multiplicity (0). For GProM and SmokedDuck, we materialize the results with the provenance information, and join the materialized results with the multiplicity table to compute the query result delta. For ProvSQL, we use the **sr_boolean()** function to evaluate the polynomial expressions under a provenance mapping setting delete tuple variables to zero. Fig. 5i and 5j shows the runtime varying the amount of deleted inputs. The percentages above the bars indicate the proportion of total execution time spent on incremental maintenance using the materialized provenance (omitted for ProvSQL as the system is not competitive). ProvSQL has significantly higher runtime cost, with the runtime keeping relatively stable across different deletion ratios, because regardless of the amount of deleted provenance, the **sr_boolean()** function always evaluates the full polynomial to decide which result tuples are affected. Thus, the cost is determined by the size of provenance expressions rather than the proportion of deleted provenance. For GProM and SmokedDuck, the materialization cost remains constant, while the cost of incremental maintenance scales in the number of deleted inputs.

Discussion. Unsurprisingly, capturing provenance that is just expressive enough to support a use-case is beneficial, showing a limitation of all four systems in not supporting direct capture of witnesses. SmokedDuck is highly efficient when only first-phase capture is considered, making it attractive when provenance must

[6]Some versions of SmokedDuck do support backtracking provenance in the second phase of capture for individual result rows, but there are limitations in the current implementation, see Sec. B.1.



(a) T1: Backward Debugging - end-to-end T_{e2e} - varying $P_{\%|Q(D)|}$ (SF1)

(b) T1: Backward Debugging - breakdown: T_{cap}, T_{query}, T_{task} (SF1)

(c) T2: Forward debug - Aggregation - $P_{\%|D|}$ 5% (SF1)

(d) T2: Forward debug - Aggregation - $P_{\%|D|}$ 5% (SF10)

(e) T3: Reprod - Full lineage (SF1)

(f) T3: Reprod - Full lineage (SF10)

(g) T3: Reprod - $P_{wit/lin}$ 0.01% (SF1)

(h) T3: Reprod - $P_{wit/lin}$ 0.01% (SF10)

(i) T4: What-If - Q_{What} - Varying r (SF1)

(j) T4: What-If - Q_{What} - Varying r (SF10)

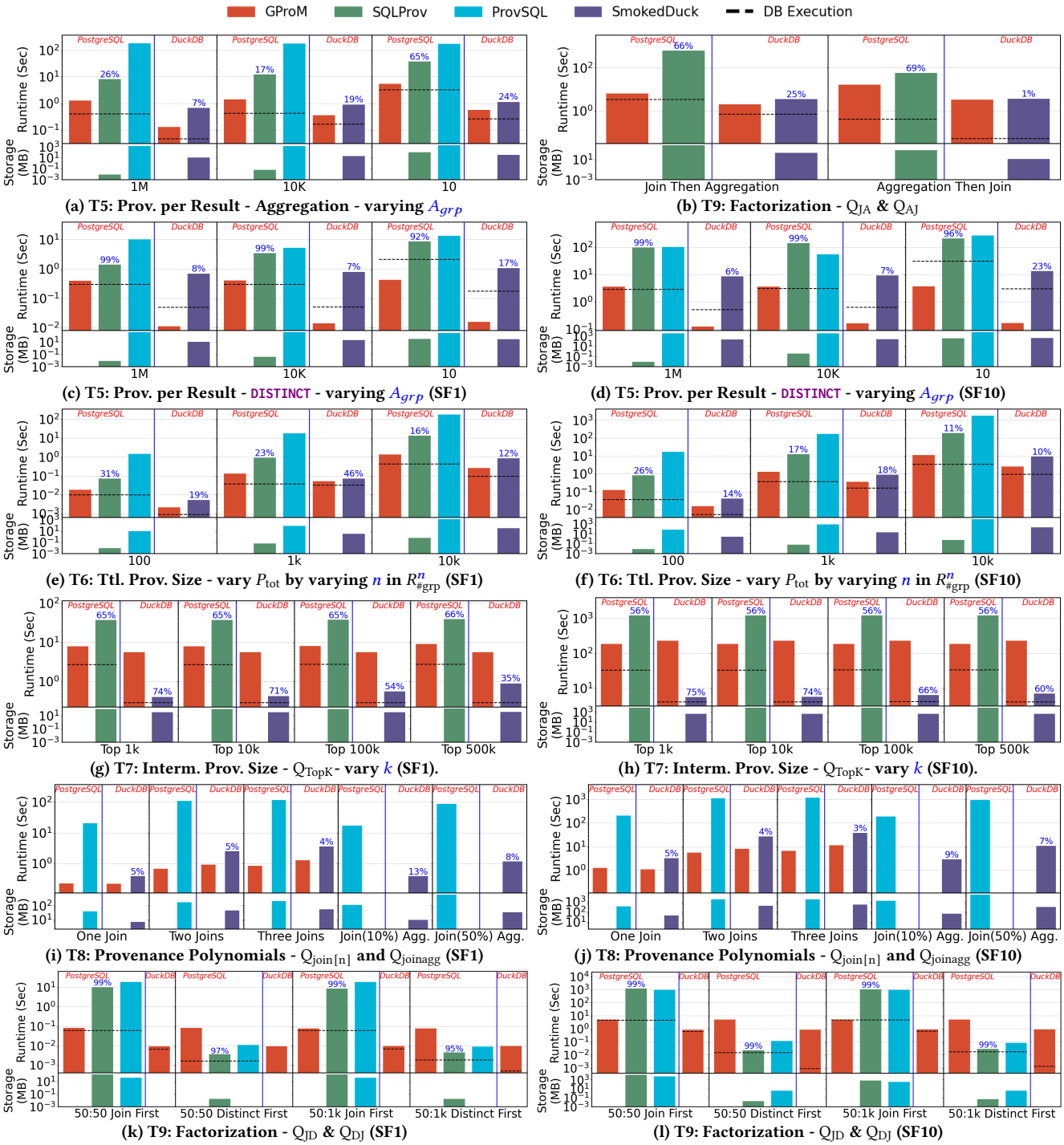
Figure 5: Experimental Evaluation for Use-case Specific Tasks

be collected eagerly for many queries. However, tasks that require reconstructed provenance can shift the advantage toward single-phase systems such as GProM. These experiments unearth an often ignored cost: many tasks require access to full row data which is surprisingly expensive for certain provenance representations.

7.3 Provenance-centric Workloads

These tasks focus exclusively on provenance capture. As usual, we show the relative time spend in the first phase for SQLProv and SmokedDuck as percentages on top of bars. As a reference, we also report the runtime of each query without provenance capture.

T5: Prov. per Result. Fig. 6c reports runtime and storage overhead while varying the provenance size per result (P_{res}), fixing total provenance size (P_{tot}). We evaluate both the aggregation query Q_{FPAgg} and the `DISTINCT` query Q_{FPDist}. Since ProvSQL's circuit-construction cost is dominated by the total number of gates in a circuit, its runtime is largely insensitive to how provenance is distributed across results. When only considering its first phase, SmokedDuck is the fastest system for Q_{FPAgg}. For both SmokedDuck and GProM, reducing P_{res} increases absolute runtime, but the relative runtime overhead over the query without provenance (dashed line) decreases. For Q_{FPDist}, GProM outperforms SmokedDuck due to employing an optimization for `DISTINCT` queries: when `DISTINCT` is the last operator, it can sometimes be removed for capture.



(a) T5: Prov. per Result - Aggregation - varying A_{grp}

(b) T9: Factorization - Q_{JA} & Q_{AJ}

(c) T5: Prov. per Result - `DISTINCT` - varying A_{grp} (SF1)

(d) T5: Prov. per Result - `DISTINCT` - varying A_{grp} (SF10)

(e) T6: Ttl. Prov. Size - vary P_{tot} by varying n in $R^{n}_{\#grp}$ (SF1)

(f) T6: Ttl. Prov. Size - vary P_{tot} by varying n in $R^{n}_{\#grp}$ (SF10)

(g) T7: Interm. Prov. Size - Q_{TopK}- vary k (SF1).

(h) T7: Interm. Prov. Size - Q_{TopK}- vary k (SF10).

(i) T8: Provenance Polynomials - $Q_{join[n]}$ and $Q_{joinagg}$ (SF1)

(j) T8: Provenance Polynomials - $Q_{join[n]}$ and $Q_{joinagg}$ (SF10)

(k) T9: Factorization - Q_{JD} & Q_{DJ} (SF1)

(l) T9: Factorization - Q_{JD} & Q_{DJ} (SF10)

Figure 6: Experimental evaluation for Provenance Structure & Size Tasks

T6: Ttl. Prov. Size. Fig. 6e and 6f shows the runtime when varying the total provenance size P_{tot} by varying the total number of groups in an aggregation query. All systems scale roughly linearly with total provenance size, indicating that total provenance volume is a primary driver of capture cost. Varying group size yields similar trends (see Sec. D.2).

T7: Interm. Prov. Size. Fig. 6g evaluates the impact of intermediate provenance by varying $P_{surviving}$, the ratio between intermediate and final provenance size, using queries with a `LIMIT` clause. As

the LIMIT threshold increases, SmokedDuck's total runtime also increases, while its first-phase capture cost remains nearly constant. This behavior reflects its two-phase design: the first phase records provenance mappings for all intermediate results, whereas the second phase reconstructs provenance top-down only for the returned tuples. Consequently, SmokedDuck benefits when the final provenance is much smaller than the intermediate provenance. In contrast, GProM and SQLProv maintain nearly constant runtime across all settings, as they use a bottom-up provenance tracing strategy which generates provenance for the large set of intermediate results only to then filter most of this provenance at the LIMIT operator. Thus, their runtime is not affected much when varying the LIMIT threshold.

T8: Provenance Polynomials. Fig. 6i shows the runtime of capturing provenance polynomials. For GProM, we request a string encoding of polynomials using WITH SEMIRING COMBINER NX to make the comparison closer to systems that explicitly materialize polynomial representations. SmokedDuck has the lowest first-phase capture time, but once reconstruction and polynomial construction are included, GProM achieves the best end-to-end runtime. ProvSQL benefits from a compact circuit encoding, but its cost is dominated by the overhead of creating circuit gates through interprocess communication and writing to memory-mapped files.

T9: Factorization. In Fig. 6b, we change the factorization structure of an aggregation over a join (Q_{JA}) by pushing the aggregation into the join (Q_{AJ}), thus, factorizing the provenance. Recall that P_{fact} is the size of factorized versus non-factorized provenance. For Q_{AJ}, P_{fact} is $\sim 1/10$. SQLProv and SmokedDuck benefit significantly from factorization, up to $\sim 12X$ faster, because compared with Q_{JA}, there is less data to be materialized in the first phase. GProM performs the worst, since it cannot exploit factorization and Q_{AJ} introduces another level of aggregation computation, making it more expensive to compute than Q_{JA}. Fig. 6k and 6l show the result for distinct after join (Q_{JD}) versus pushing the duplicate elimination (Q_{DJ}) for $P_{fact} \simeq \frac{1}{200}$ and $P_{fact} = \frac{1}{20}$. SmokedDuck is excluded as the provenance it produces for DISTINCT is incomplete. While GProM's runtime is nearly identical for the two queries because it cannot exploit factorization, SQLProv and SmokedDuck capitalize on the dramatically smaller provenance representation. For SF10 ProvSQL's representation of circuits as individual gates pays off and it outperforms SQLProv which generates large arrays.

Discussion. These experiments show that the choice of capture strategy has a major impact on performance. When only sufficient information needs to be recorded for later provenance reconstruction, two-phase systems have a clear advantage because they execute only the first capture phase. In this setting, SmokedDuck often achieves the lowest overhead. However, surprisingly, when retrieval of provenance is also taken into account then SmokedDuck is outperformed by GProM which is due to its high indexing cost of captured provenance. Exploiting provenance factorization can substantially reduce runtime and storage requirements. However, these benefits are only significant when the provenance is highly factorizable. For moderate levels of factorization, the higher per-tuple overhead of SQLProv and ProvSQL outweigh the gains.

	PostgreSQL						DuckDB			
	GProM	SQLProv			ProvSQL		GProM	SmokedDuck		
	T_{cap}	T_{log}	T_{trace}	S_{cap}	T_{cap}	S_{cap}	T_{cap}	T_{log}	T_{trace}	S_{cap}
Q_1	**2.90**	28.20	TO	327.6	TO	TO	0.910	**0.160**	**0.386**	22.7
Q_2	0.27	**0.12**	**0.01**	0.1	S	S	**0.183**	0.014	0.354	14.0
Q_3	**0.47**	0.80	0.20	2.6	S	S	0.178	**0.061**	**0.091**	20.0
Q_4	2.20	**0.50**	**0.16**	2.9	S	S	**0.352**	0.062	0.340	18.7
Q_5	**0.27**	0.40	0.10	0.5	S	S	**0.177**	0.070	0.123	5.4
Q_6	**0.31**	1.30	1.67	6.3	2.01	75.75	0.071	**0.032**	**0.002**	0.5
Q_7	0.97	0.64	0.08	0.4	**0.64**	25.81	**0.336**	0.150	0.783	72.3
Q_8	**0.20**	S	S	S	S	S	0.485	**0.042**	**0.083**	4.0
Q_9	**0.59**	3.08	3.40	21.9	8.5	292	2.409	**0.153**	**0.252**	15.6
Q_{10}	**0.80**	1.62	0.73	9.4	S	S	0.389	**0.123**	**0.104**	16.7
Q_{11}	1.65	**0.37**	**0.02**	4.0	S	S	6.272	**0.008**	**0.030**	1.1
Q_{12}	**0.43**	S	S	S	2.55	50.8	0.125	**0.061**	**0.051**	4.7
Q_{13}	**1.60**	11.50	68.62	98.0	S	S	**0.643**	0.168	0.533	41.1
Q_{14}	**0.31**	S	S	S	S	S	0.064	**0.035**	**0.027**	7.2
Q_{15}	1.75	**1.29**	**0.15**	1.3	S	S	1.490	**0.058**	**0.070**	5.6
Q_{16}	TO	**1.08**	**0.37**	8.5	S	S	TO	**0.022**	**0.050**	3.4
Q_{17}	**3.06**	4.20	0.01	10.4	S	S	0.512	**0.055**	**0.118**	0.6
Q_{18}	4.31	**2.15**	**0.01**	0.1	S	S	TO	**0.361**	**1.462**	159.9
Q_{19}	0.04	0.05	0.01	0.1	**0.09**	24.3	**0.141**	0.071	0.075	14.6
Q_{20}	S	S	S	S	S	S	S	**0.055**	**0.033**	4.2
Q_{21}	S	**0.93**	**0.05**	0.3	S	S	S	**0.177**	**0.457**	46.0
Q_{22}	26.32	**0.10**	**0.01**	0.4	S	S	TO	**0.031**	**0.009**	3.9

Figure 7: SF1

	PostgreSQL						DuckDB			
	GProM	SQLProv			ProvSQL		GProM	SmokedDuck		
	T_{cap}	T_{log}	T_{trace}	S_{cap}	T_{cap}	S_{cap}	T_{cap}	T_{log}	T_{trace}	S_{cap}
Q_1	**12.5**	TO	TO	S	TO	TO	**1.762**	1.511	4.587	226
Q_2	2.23	**1.92**	**0.003**	0.4	S	S	16.61	**0.172**	**3.891**	139
Q_3	**7.16**	14.2	1.73	30	S	S	**0.952**	0.805	1.012	198
Q_4	**4.77**	5.35	0.18	33.5	S	S	13.56	**0.652**	**3.655**	187
Q_5	7.71	**5.30**	**0.81**	5.8	S	S	**1.409**	0.874	1.762	49.5
Q_6	**4.55**	11.1	2.25	74	21.5	4178	**0.329**	0.322	0.026	5.23
Q_7	**6.05**	12.9	0.68	4.7	11.6	3634	**2.600**	2.157	9.512	810
Q_8	**2.07**	S	S	S	S	S	26.09	**0.881**	**1.101**	38.9
Q_9	**13.2**	36.1	36.5	262	91.3	2642	**2.290**	2.000	3.460	158
Q_{10}	**8.01**	19.0	5.82	110	S	S	3.423	**1.302**	**1.197**	140
Q_{11}	TO	**4.14**	**0.08**	48	S	S	TO	**0.083**	**0.290**	10.7
Q_{12}	**6.39**	S	S	S	25.3	4146	**0.606**	0.606	0.503	47.1
Q_{13}	**18.4**	X	X	X	S	S	**7.374**	2.098	7.696	411
Q_{14}	**6.23**	S	S	S	S	S	**0.364**	0.460	0.444	35.2
Q_{15}	22.81	**11.1**	**1.96**	14.7	S	S	9.366	**0.590**	**0.739**	56.0
Q_{16}	TO	**11.7**	**3.06**	84.6	S	S	TO	**0.259**	**0.616**	29.2
Q_{17}	44.1	**41.0**	**0.05**	124	S	S	8.875	**1.197**	**1.335**	5.20
Q_{18}	37.6	**28.4**	**0.04**	0.46	S	S	TO	**5.105**	**19.07**	1627
Q_{19}	**0.55**	0.63	0.01	0.08	0.72	2098	**0.743**	0.824	1.158	115
Q_{20}	S	S	S	S	S	S	S	**0.569**	**0.349**	42.2
Q_{21}	S	**9.11**	**0.40**	3.4	S	S	S	**2.101**	**5.243**	455
Q_{22}	TO	**1.21**	**0.13**	4.1	S	S	TO	**0.351**	**0.103**	38.6

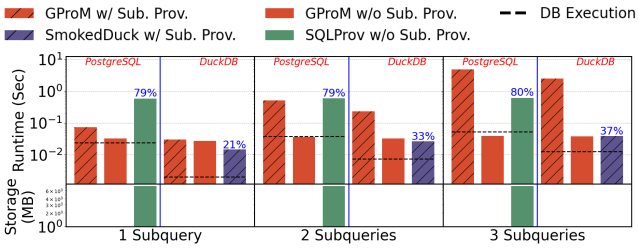
Figure 8: SF10

Figure 9: T10: TPC-H: runtime (T_{cap} in sec.) and storage overhead (S_{cap} in MB). T_{log} (T_{trace}) is the runtime for phase I (II).

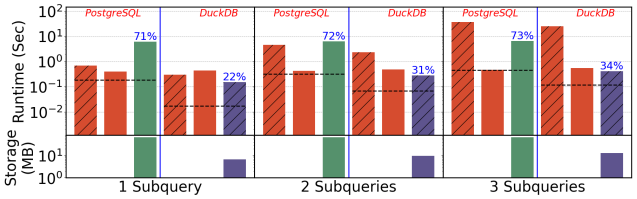
7.4 Query-centric Workloads

We finally evaluate workloads that stress query structure and SQL language features. As in the provenance-centric workloads, we report only T_{log} for two-phase systems such as SmokedDuck and SQLProv, unless noted otherwise.

7.4.1 TPC-H. **T10: TPC-H.** Fig. 7 shows provenance capture runtime for *TPC-H* queries. We set a timeout of 5 minutes. We use the same indicators as in sec. 6 (S) to indicate that a system does not suport a query. TO indicates that the system timed out. The fastest system's runtime for a query and DBMS is highlighted in bold. For several queries, provenance size is moderate, making SQLProv and ProvSQL more competitive with the exception of Q_1 and Q_{13}, where provenance size is large (e.g., ProvSQL writes over $\sim$ 2.5 GB of data to disk for query Q_1). Excluding queries with nested



(a) SF1



(b) SF10

Figure 10: T11: Sub-Q - Vary number of subqueries in Q_{WSub}

subqueries, SmokedDuck is up to ~ 7 times faster than GProM and 2-3 times faster for most queries. Furthermore, for queries with (multiple) nested subqueries (e.g., Q_2, Q_{11}, Q_{17}, Q_{21}, and Q_{22}) the capability of a system to factorize provenance is critical: in fact, for some of these queries SmokedDuck outperforms GProM by several OOM. GProM times out on more queries for DuckDB, even though DuckDB is generally faster than PostgreSQL, due to having to use DuckDB 1.0.0 and no parallelism ($threads = 1$) for compatibility with SmokedDuck.

T11: Sub-Q. Fig. 10 shows the result for Q_{WSub}, varying the number of subqueries in the `WHERE` clause. For GProM and SmokedDuck, capturing provenance for tables inside subqueries causes runtime to grow rapidly, because each additional subquery increases the total provenance size by roughly an order of magnitude. When provenance is not captured for tables appearing inside subqueries, all evaluated systems remain comparatively stable, since the subqueries primarily act as a filter only.

Discussion. The query-centric workloads show that query structure and SQL constructs can be as important as provenance size. For complex queries with relatively small provenance, ProvSQL and SQLProv are closer to GProM, typically within a small constant factor rather than orders of magnitude slower. Nested subqueries expose important differences in representation and capture strategy. GProM suffers from rewriting nested subqueries into LATERAL joins, which are not always optimized well by PostgreSQL, and from using a flat, non-factorized representation that cannot exploit factorization and enumerates all combinations of tuples drawn from the provenance of each subquery. These results reinforce the need for query-centric workloads in PBench.

8 RELATED WORK

Provenance Models. Several provenance models have been proposed to provide a formal semantics for explaining query results under different classes of queries [14, 22, 31]. A witness [11] is a subset of database that is sufficient to generate the same result. Lineage [17, 19], capturing all inputs that contribute to a result of query, was first introduced in [60] and then was formalized based on sufficiency and necessity [19]. Why-provenance [11, 19] identifies which sets of input tuples (witnesses) are sufficient to generate the result. Where-provenance [11] traces the input tuples from which the result is derived. Green et al. [26, 27] introduced K-relations and demonstrated that the semiring $\mathbb{N}[X]$ whose elements are polynomials over variables X that represent tuples generalizes all other provenance models expressible as semirings (including, e.g., lineage and why-provenance). Niu et al [48] proposes provenance sketches, a coarse-grained provenance model. The approach captures sketches over-approximating the provenance of a query and utilizes these sketches to speed-up subsequent queries. [2, 3] discuss how to summarize provenance polynomials by merging multiple variables into one new variable. Why-not [13] methods explain missing results using provenance. Detailed introductions to relational provenance models can be found in [14, 22, 31, 34].

Provenance Use Cases. Important use cases of provenance include the creation of *"reproducibility packages"* for computational experiments [15, 57] that include sufficient input data to guarantee that computations (queries in the relational case) produce the same result as in the original experiment [50, 51]. Provenance serves as a tool for debugging. [47] presents a system for debugging transactions with provenance. Provenance is essential for probabilistic databases [54–56, 59], where provenance polynomials (or lineage formulas for set semantics) provide a symbolic representation of tuple derivations and are used to compute the marginal probabilities of query results. In view maintenance under deletion propagation scenarios [12, 35], provenance polynomials are sufficient for maintaining results as exploited, e.g., in FADE [45]. Provenance polynomials have also been used in How-to analysis [29, 41, 42, 62].

Relational Provenance Systems. Provenance can be captured by annotating data and propagating these annotations [6, 23], by recording mappings between input and output tuples of operators [44, 46, 53, 54], or by top-down backtracing results to the inputs that contributed to them [19]. This can be done by using relational queries or by extending the database system [36, 49]. Systems like GProM [6], Perm [25], Smoke [53], SmokedDuck [44], Links [21], ProvSQL[55], SQLProv [46], Trio [1], DataProv [52], and DBNotes[8] capture provenance for SQL queries.

9 CONCLUSIONS AND DISCUSSION

We presented PBench, the first comprehensive benchmark for relational provenance systems. PBench evaluates systems along three complementary dimensions: SQL-centric workloads that stress query features, provenance-centric workloads that isolate the effects of provenance size and representation, and use-case-driven workloads that measure end-to-end performance for applications such as debugging, reproducibility, and probabilistic query processing. Using PBench, we systematically compare four representative provenance systems: GProM, ProvSQL, SQLProv, and SmokedDuck.

Guidance for selecting provenance systems. Our compliance analysis and experimental results can aid practitioners in selecting the right provenance system for their needs and to estimate the resources that are required to support their provenance workloads. The first important choice is what database backend the user wants to build on as this already restricts the choice of provenance

system. Furthermore, based on the targeted use case and required SQL features, provenance systems that do not support sufficiently expressive provenance models or that lack support for the right SQL language features can be excluded. Another important consideration are storage requirements. ProvSQL and SQLProv add significant storage overhead to enable provenance capture for a database and incur additional overhead during provenance capture. Practitioners should take our results into account to determine whether they have sufficient storage available to use one of these systems for their provenance needs.

Our results show that no single system dominates across all workloads. Tight integration with the query engine minimizes provenance-capture overhead, whereas systems that expose provenance directly as relational data often provide superior end-to-end performance. More generally, our evaluation highlights the importance of minimizing the per-tuple cost of provenance capture, as lightweight capture mechanisms often outperform more sophisticated representations. We further show that provenance representation and capture strategy have a greater impact on end-to-end efficiency than capture cost alone. In particular, support for provenance factorization is essential. Overall, our study demonstrates that provenance systems should be designed and evaluated with their target applications in mind. Many practical tasks require only a subset of provenance or a simpler provenance model, making highly expressive provenance unnecessarily expensive. Conversely, applications such as view maintenance fundamentally require more expressive provenance despite their higher cost. By providing a unified methodology for evaluating provenance capture, storage, querying, and downstream use, PBench enables reproducible comparison of provenance systems and provides a foundation for designing future systems that better balance expressiveness, efficiency, and usability.

REFERENCES

[1] Parag Agrawal, Omar Benjelloun, Anish Das Sarma, Chris Hayworth, Shubha U. Nabar, Tomoe Sugihara, and Jennifer Widom. 2006. Trio: A System for Data, Uncertainty, and Lineage. In *VLDB*. 1151–1154.

[2] Eleanor Ainy, Pierre Bourhis, Susan B. Davidson, Daniel Deutch, and Tova Milo. 2015. Approximated Summarization of Data Provenance. In *CIKM*. 483–492.

[3] Eleanor Ainy, Pierre Bourhis, Susan B. Davidson, Daniel Deutch, and Tova Milo. 2016. PROX: Approximated Summarization of Data Provenance. In *EDBT*. 620–623.

[4] Shatha Algarni, Boris Glavic, Seokki Lee, and Adriane Chapman. 2025. Efficient Query Repair for Aggregate Constraints. *PVLDB* (2025).

[5] Yael Amsterdamer, Daniel Deutch, and Val Tannen. 2011. Provenance for Aggregate Queries. In *PODS*. 153–164.

[6] Bahareh Arab, Su Feng, Boris Glavic, Seokki Lee, Xing Niu, and Qitian Zeng. 2018. GProM - A Swiss Army Knife for Your Provenance Needs. *IEEE Data Eng. Bull.* 41, 1 (2018), 51–62.

[7] Sepehr Assadi, Sanjeev Khanna, Yang Li, and Val Tannen. 2016. Algorithms for Provisioning Queries and Analytics. In *ICDT*, Vol. 48. 18:1–18:18.

[8] Deepavali Bhagwat, Laura Chiticariu, Wang Chiew Tan, and Gaurav Vijayvargiya. 2005. An annotation management system for relational databases. *VLDBJ* 14, 4 (2005), 373–396.

[9] Nicole Bidoit, Melanie Herschel, and Aikaterini Tzompanaki. 2015. Efficient Computation of Polynomial Explanations of Why-Not Questions. In *CIKM*. 713–722.

[10] Nicole Bidoit, Melanie Herschel, Katerina Tzompanaki, et al. 2014. Query-Based Why-Not Provenance with NedExplain. In *EDBT*.

[11] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. 2001. Why and Where: A Characterization of Data Provenance. In *ICDT*. 316–330.

[12] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. 2002. On Propagation of Deletions and Annotations through Views. In *PODS*. 150–158.

[13] Adriane Chapman and H. V. Jagadish. 2009. Why Not?. In *SIGMOD*. 523–534.

[14] James Cheney, Laura Chiticariu, and Wang-Chiew Tan. 2009. Provenance in Databases: Why, How, and Where. *Foundations and Trends in Databases* 1, 4 (2009), 379–474.

[15] Fernando Chirigati, Rémi Rampin, Dennis Shasha, and Juliana Freire. 2016. Reprozip: Computational reproducibility with ease. In *SIGMOD*. 2085–2088.

[16] Gao Cong, Wenfei Fan, Floris Geerts, Jianzhong Li, and Jizhou Luo. 2012. On the Complexity of Annotation Propagation and View Update Analyses. *IEEE TKDE* 24, 3 (2012), 506–519.

[17] Yingwei Cui and Jennifer Widom. 2001. Lineage Tracing for General Data Warehouse Transformations. In *VLDB*. 471–480.

[18] Yingwei Cui and Jennifer Widom. 2001. *Run-time translation of view tuple deletions using data lineage.* Technical Report.

[19] Yingwei Cui, Jennifer Widom, and Janet L. Wiener. 2000. Tracing the Lineage of View Data in a Warehousing Environment. *TODS* 25, 2 (2000), 179–227.

[20] Ralf Diestelkämper, Seokki Lee, Melanie Herschel, and Boris Glavic. 2021. To not miss the forest for the trees - A holistic approach for explaining missing answers over nested data. In *SIGMOD*. 405–417.

[21] Stefan Fehrenbach and James Cheney. 2018. Language-integrated provenance. *Sci. Comput. Program.* 155 (2018), 103–145.

[22] Boris Glavic. 2021. Data Provenance - Origins, Applications, Algorithms, and Models. *Foundations and Trends® in Databases* 9, 3-4 (2021), 209–441.

[23] Boris Glavic and Gustavo Alonso. 2009. Perm: Processing Provenance and Data on the same Data Model through Query Rewriting. In *ICDE*. 174–185.

[24] Boris Glavic, Pengyuan Li, Ziyu Liu, Dieter Gawlick, Vasudha Krishnaswamy, Danica Porobic, and Zhen Hua Liu. 2024. Towards an Objective Metric for Data Value Through Relevance. In *CIDR*.

[25] Boris Glavic, Renée J. Miller, and Gustavo Alonso. 2013. Using SQL for Efficient Generation and Querying of Provenance Information. In *In Search of Elegance in the Theory and Practice of Computation - Essays Dedicated to Peter Buneman*, Vol. 8000. 291–320.

[26] Todd J. Green, Gregory Karvounarakis, and Val Tannen. 2007. Provenance Semirings. In *PODS*. 31–40.

[27] Todd J Green and Val Tannen. 2017. The Semiring Framework for Database Provenance. In *PODS*. 93–99.

[28] Erich Grädel and Val Tannen. 2017. Semiring Provenance for First-Order Model Checking. *arXiv preprint arXiv:1712.01980* (2017).

[29] Erich Grädel and Val Tannen. 2020. Provenance analysis for logic and games. *Moscow Journal of Combinatorics and Number Theory* 9, 3 (2020), 203–228.

[30] Ashish Gupta, Inderpal Singh Mumick, and Venkatramanan Siva Subrahmanian. 1993. Maintaining views incrementally. In *SIGMOD Record*, Vol. 22. 157–166.

[31] Melanie Herschel, Ralf Diestelkämper, and Houssem Ben Lahmar. 2017. A survey on provenance: What for? What form? What from? *VLDBJ* (2017), 1–26.

[32] Rinke Hoekstra and Paul Groth. 2014. PROV-O-Viz-understanding the role of activities in provenance. In *International Provenance and Annotation Workshop*. 215–220.

[33] Xiao Hu and Stavros Sintos. 2024. Finding Smallest Witnesses for Conjunctive Queries. In *ICDT*.

[34] Grigoris Karvounarakis and Todd J. Green. 2012. Semiring-Annotated Data: Queries and Provenance. *SIGMOD Record* 41, 3 (2012), 5–14.

[35] Benny Kimelfeld, Jan Vondrák, and Ryan Williams. 2012. Maximizing Conjunctive Views in Deletion Propagation. *TODS* 37, 4 (2012), 24:1–24:37.

[36] Sven Köhler, Bertram Ludäscher, and Yannis Smaragdakis. 2012. Declarative datalog debugging for mere mortals. In *International Datalog 2.0 Workshop*. 111–122.

[37] Seokki Lee, Sven Köhler, Bertram Ludäscher, and Boris Glavic. 2016. Implementing Unified Why- and Why-Not Provenance Through Games. In *TaPP*.

[38] Seokki Lee, Bertram Ludäscher, and Boris Glavic. 2018. PUG: a framework and practical implementation for why and why-not provenance. *VLDBJ* 28, 1 (2018), 47–71.

[39] Jinyang Li, Yuval Moskovitch, Julia Stoyanovich, and H. V. Jagadish. 2023. Query Refinement for Diversity Constraint Satisfaction. *PVLDB* 17, 2 (2023), 106–118.

[40] Jinyang Li, Alon Silberstein, Yuval Moskovitch, Julia Stoyanovich, and H. V. Jagadish. 2023. ERICA: Query Refinement for Diversity Constraint Satisfaction. *PVLDB* 16, 12 (2023), 4070–4073.

[41] Alexandra Meliou, Wolfgang Gatterbauer, and Dan Suciu. 2011. Reverse data management. *PVLDB* 4, 12 (2011).

[42] Alexandra Meliou and Dan Suciu. 2012. Tiresias: The database oracle for how-to queries. In *SIGMOD*. 337–348.

[43] Chaitanya Mishra and Nick Koudas. 2009. Interactive query refinement. In *EDBT*. 862–873.

[44] Haneen Mohammed, Charlie Summers, Sughosh V. Kaushik, and Eugene Wu. 2023. SmokedDuck Demonstration: SQLStepper. In *SIGMOD*. 183–186.

[45] Haneen Mohammed, Eugene Wu, Alexander Yao, Charlie Summers, Lampros Flokas, Gromit Yeuk-Yin Chan, Subrata Mitra, and Hongbin Zhong. 2024. Fade: More Than a Million What-Ifs Per Second. *PVLDB* 18, 4 (2024), 943–955.

[46] Tobias Müller, Benjamin Dietrich, and Torsten Grust. 2018. You say'what', i hear'where'and'why':(mis-) interpreting SQL to derive fine-grained provenance. *PVLDB* 11, 11 (2018), 1536–1549.

[47] Xing Niu, Boris Glavic, Seokki Lee, Bahareh Arab, Dieter Gawlick, Zhen Hua Liu, Vasudha Krishnaswamy, Su Feng, and Xun Zou. 2017. Debugging Transactions and Tracking their Provenance with Reenactment. *PVLDB* 10, 12 (2017), 1857–1860.

[48] Xing Niu, Boris Glavic, Ziyu Liu, Pengyuan Li, Dieter Gawlick, Vasudha Krishnaswamy, Zhen Hua Liu, and Danica Porobic. 2021. Provenance-based Data Skipping. *PVLDB* 15, 3 (2021), 451–464.

[49] Xing Niu, Raghav Kapoor, Boris Glavic, Dieter Gawlick, Zhen Hua Liu, and Venkatesh Radhakrishnan. 2017. Provenance-Aware Query Optimization. In *ICDE*. 473–484.

[50] Quan Pham, Tanu Malik, Boris Glavic, and Ian Foster. 2015. LDV: Light-weight Database Virtualization. In *ICDE*. 1179–1190.

[51] Quan Pham, Richard Whaling, Boris Glavic, and Tanu Malik. 2015. Sharing and Reproducing Database Applications. *PVLDB* 8, 12 (2015), 1988–1999.

[52] Paulo Pintor, Rogério Luís de C. Costa, and José Manuel Moreira. 2026. A Dbms-Independent Approach for Capturing Provenance Polynomials Through Query Rewriting. *IEEE Access* 14 (2026), 66114–66133.

[53] Fotis Psallidas and Eugene Wu. 2018. Smoke: Fine-grained Lineage at Interactive Speed. *PVLDB* 11, 6 (2018), 719–732.

[54] Aryak Sen, Silviu Maniu, and Pierre Senellart. 2026. ProvSQL: A General System for Keeping Track of the Provenance and Probability of Data. In *ICDE*.

[55] Pierre Senellart, Louis Jachiet, Silviu Maniu, and Yann Ramusat. 2018. ProvSQL: Provenance and Probability Management in PostgreSQL. *PVLDB* 11, 12 (2018), 2034–2037.

[56] Dan Suciu. 2020. Probabilistic databases for all. In *PODS*. 19–31.

[57] Dai Hai Ton That, Gabriel Fils, Zhihao Yuan, and Tanu Malik. 2017. Sciunits: Reusable research objects. *arXiv preprint arXiv:1707.05731* (2017).

[58] Quoc Trung Tran and Chee-Yong Chan. 2010. How to ConQueR why-not questions. In *SIGMOD*. 15–26.

[59] Guy Van den Broeck, Dan Suciu, et al. 2017. Query processing on probabilistic data: A survey. *Foundations and Trends® in Databases* 7, 3-4 (2017), 197–341.

[60] Allison Woodruff and Michael Stonebraker. 1997. Supporting Fine-grained Data Lineage in a Database Visualization Environment. In *ICDE*. 91–102.

[61] Eugene Wu, Fotis Psallidas, Zhengjie Miao, Haoci Zhang, and Laura Rettig. 2017. Combining Design and Performance in a Data Visualization Management System. In *CIDR*.

[62] Jane Xu, Waley Zhang, Abdussalam Alawini, and Val Tannen. 2018. Provenance Analysis for Missing Answers and Integrity Repairs. *Data Engineering* (2018), 39.

A QUERY TEMPLATES

In this section, we present all query templates used in the benchmark tasks.

Task	Query	Tables	Parameter	Join Format	Tot. Prov.	Inter. Res.	Res.
T1: Backward Debugging	Q_{BDebug}	$R^{1k}_{\#grp}, R^{c}_{join}$	–	**1000%(1 – 10)**	~ 1M	10M	1K
T2: Forward debug	Q_{FD1}	$R^{1k}_{\#grp}, R^{c}_{join}$	–	**1000%(1 – 10)**	~ 0.33M	10M	333
	Q_{FD2}	$R^{1k}_{\#grp}, R^{c}_{join}$	–	**1000%(1 – 10)**	~ 0.33M	10M	~ 3.3M
T??: ??	Q_{Lin}	$R^{10M}_{fp}, R^{c}_{join}$	–	**1000%(1 – 10)**	~ 10M	100M	1K
T??: ??	Q_{Wit}	$R^{n}_{\#grp}, R^{c}_{join}$	$n = 1K$	**1000%(1 – 10)**	~ 11K	10K	1K
			n = 100K	**1000%(1 – 10)**	~ 101K	1M	1K
			n = 2M	**1000%(1 – 10)**	~ 2M	20M	1K
T8: Provenance Polynomials	$Q_{join[1]}$	$R^{1k}_{\#grp}, R^{c}_{join}$	–	**100%(1[10%] – 10)**	~ 0.1M	1M	1M
	$Q_{join[2]}$	$R^{1k}_{\#grp}, R^{c}_{join}, R^{j}_{join}$	–	**500%(1[10%] – 10 – 5)**	~ 0.1M	5M	5M
	$Q_{join[3]}$	$R^{1k}_{\#grp}, R^{c}_{join}, R^{j}_{join}, R^{jj}_{join}$	–	**500%(1[10%] – 10 – 5 – 1)**	~ 0.1M	5M	5M
T5: Prov. per Result	$Q_{joinagg}$	$R^{1k}_{\#grp}, R^{s}_{join}$	A_{sel} = p10	**100%(1[10%] – 10)**	~ 0.1M	1M	100
			A_{sel} = p50	**500%(1[50%] – 10)**	~ 0.5M	5M	500
	Q_{FPAgg}, Q_{FPDist}	R^{10M}_{fp}	$\|A_{grp}\| \in \{10, 1K, 1M\}$	–	10M	–	$\{10, 1K, 1M\}$
T6: Ttl. Prov. Size	Q_{VPGN}	$R^{n}_{\#grp}$	$n \in \{100, 1K, 10K\}$	–	{0.1M, 1M, 10M}	–	$\{100, 1K, 1M\}$
	Q_{VPGS}	$R^{n}_{grpsize}$	$n \in \{100, 1K, 10K\}$	–	{0.1M, 1M, 10M}	–	$1K$
T7: Interm. Prov. Size	Q_{TopK}	R^{10M}_{fp}	$k \in$ {1K, 10K, 100K, 500K}	–	~ 10M	1M	k
T11: Sub-Q	Q_{WSub}	$R^{100}_{\#grp}, R^{c}_{join}$	$n \in \{1, 2, 4\}$	–	~ 100K	{1 M, 10 M, 100 M}	100

Table 6: Query templates ($P_{SF} = 1$) discussed in Sec. A: tables accessed, varied parameters, join format, total provenance size (Tot. Prov.), intermediate result size (Inter. Res.) and result size (Res.)

A.1 Use-case Specific Tasks

A.1.1 *Backward Debugging.*

Q_{BDebug} shown below is used in T1: Backward Debugging:

```
SELECT ga, avg(va) AS ava, avg(vb) AS avb
FROM R^{1k}_{#grp} JOIN R^{c}_{join} ON (ga = c1to10)
GROUP BY ga
```

Q_{BDebug} joins each tuple from $R^{1k}_{\#grp}$ with 10 tuples from R^{c}_{join}, producing a total of $1K$ result tuples. The total provenance size is $\sim 1M \cdot P_{SF}$ $\left(1P_{SF} \cdot (M + 10K)\right)$. The provenance of each result tuple contains $\sim 10K \cdot P_{SF}$ tuples. In this task, we vary the fraction of result tuples for which provenance needs to be produced.

A.1.2 *Forward Debugging.*

Q_{FD1} shown below is used in T2: Forward debug:

```
SELECT ga, avg(va) AS ava, avg(vb) AS avb
FROM R^{1k}_{#grp}, R^{c}_{join}
WHERE ga = c1to10 and ga >= 667 and ga <= 1000
GROUP BY ga
```

Q_{FD1} joins each tuple in $R^{1k}_{\#grp}$ with 10 tuples in R^{c}_{join} and produces 334 result tuples with $3.34M \cdot P_{SF}$ intermediate tuples generated by the join operator. The total size of provenance is $\sim 0.33M \cdot P_{SF}$.

Q_{FD2} is used in T2: Forward debug and T14: Forward debugging - provenance and result:

```
SELECT ga, va, vb
FROM R^{1k}_{#grp}, R^{c}_{join}
WHERE ga = c1to10
  AND ga >= 667
  AND ga <= 1000
```

Both the final result and intermediate provenance sizes of Q_{FD2} are $3.34M \cdot P_{SF}$. The total size of provenance is the same as Q_{FD1}. Both join operators in these query templates have a 100% selectivity and each tuple from $R^{1k}_{\#grp}$ has 10 join partners in table R^{c}_{join}.

A.1.3 *Reproducibility.*

Q_{Lin} (shown below) is used in T??: ??: Query Q_{Lin} accesses R^{10M}_{fp} and R^{c}_{join}, and outputs $1K$ result tuples. The join selectivity is 100% and each tuple from R^{10M}_{fp} can join 10 tuples from R^{c}_{join}. The intermediate tuple size is $100M \cdot P_{SF}$ and the total provenance size is $\sim 10M \cdot P_{SF}$.

Q_{Wit} (shown below) is used in T??: ??. Q_{Wit} uses R^{n}_{fp}, reduced size of R^{10M}_{fp} but total number of groups kept the same, varying $n \in \{1K \cdot P_{SF}, 100K \cdot P_{SF}, 2M \cdot P_{SF}\}$ which is the total size of the table. The result sizes are $1K \cdot P_{SF}$ even if we vary the n, and the total provenance sizes are in $\{11K \cdot P_{SF}, 101K \cdot P_{SF}, \sim 2M \cdot P_{SF}\}$. The intermediate result sizes are in $\{110K \cdot P_{SF}, \sim 1M \cdot P_{SF}, \sim 10M \cdot P_{SF}\}$.

Q_{Lin}:
```
SELECT gb, min(va) AS minva
FROM R^{10M}_{fp} JOIN R^{c}_{join} ON gb = c1to10
GROUP BY gb
```

Q_{Wit}:
```
SELECT gb, min(va) AS minva
FROM R^{n}_{fp} JOIN R^{c}_{join} ON gb = c1to10
GROUP BY gb
```

A.1.4 *Provenance Polynomials.*

The three SPJ queries, $Q_{join[1]}, Q_{join[2]}, Q_{join[3]}$, varying the number of join operations, and one SPJA qury,$Q_{joinagg}$ varying the join selectivity, are are used for T8: Provenance Polynomials, show as below:

$Q_{join[1]}$:
```
SELECT ga
FROM R^{1k}_{#grp} JOIN R^{s}_{join} ON ga = p10
```

$Q_{join[2]}$:
```
SELECT ga
FROM R^{1k}_{#grp} JOIN R^{s}_{join} ON ga = p10 JOIN R^{j}_{join} ON ga = c5
```

$Q_{join[3]}$:
```
SELECT ga
FROM R^{1k}_{#grp}
     JOIN R^{s}_{join} ON ga = p10
     JOIN R^{j}_{join} ON ga = c5
     JOIN R^{jj}_{join} ON ga = cc1
```

$Q_{joinagg}$:
```
SELECT ga, avg(va) AS ava
FROM R^{1k}_{#grp} JOIN R^{s}_{join} ON ga = A_{sel}
GROUP BY ga
```

For SPJ queries, $Q_{join[1]}$, $Q_{join[2]}$ and $Q_{join[3]}$, we vary the number of join operators to vary the intermediate size in terms of returned rows in $\{1M \cdot P_{SF}, 5M \cdot P_{SF}, 5M \cdot P_{SF}\}$, leading result sizes of $1M \cdot P_{SF}$,

5M · P_{SF} and 5M · P_{SF}. The total size of provenance tuples is ∼ 0.1M · P_{SF} for all queries. For $Q_{join[2]}$ and $Q_{join[3]}$, even through the size of the total provenance and intermediate results and the final result are the same, the size of polynomials of each result tuple differ: 3 and 4, respectively. One rationale for controlling the number of join operators is to examine systems' behavior on different polynomial structure, fixing all other settings.

$Q_{joinagg}$ accesses tables $R^{1K}_{\#grp}$ and R^{s}_{join} varying the join selectivity in {10%, 50%}, controlled by selecting the attribute A_{sel} in R^{s}_{join} to join, yielding intermediate result sizes of 1M · P_{SF} and 5M · P_{SF} and final result sizes of 100 and 500. The size of the total provenance is then ∼ 0.1M · P_{SF} and ∼ 0.5M · P_{SF}.

A.2 Provenance Structure & Size

A.2.1 Provenance Size per Result Tuple.
Q_{FPAgg} and Q_{FPDist} are used for T5: Prov. per Result.

Q_{FPAgg}:
```
SELECT A_grp, avg(va) AS ava, avg(vb) AS avb
FROM R^10M_fp
GROUP BY A_grp
```

Q_{FPDist}:
```
SELECT DISTINCT A_grp
FROM R^10M_fp
```

Both queries access a single table R^{10M}_{fp}. We control the number of groups (or distinct values) to $P_{SF} \cdot 10(1K or 1M)$ by selecting which attribute $P_{A_{grp}}$ to group on (or project out in case of distinct) for the purpose of varying provenance size per result tuple.

A.2.2 Varying Total Provenance Size.
Q_{VPGN} and Q_{VPGS} show below are used in T6: Ttl. Prov. Size to vary the total provenance size. Q_{VPGN} fixes the numer of tuples per group, $1K \cdot P_{SF}$, and vary the number of groups $n \in \{100, 1K, 10K\}$, and Q_{VPGS} varies the size of per group:

$$n \in \{100, 1K, 10K\},$$

keeping the number of groups fixed to $1K \cdot P_{SF}$.

Q_{VPGN}:
```
SELECT ga, avg(va) AS ava, avg(vb) AS avb
FROM R^n_#grp
GROUP BY ga
```

Q_{VPGS}:
```
SELECT ga, avg(va) AS ava, avg(vb) AS avb
FROM R^n_grpsize
GROUP BY ga
```

The total provenance sizes for both tables ranges in

$$\{100K \cdot P_{SF}, 1M \cdot P_{SF}, 10M \cdot P_{SF}\}.$$

A.2.3 Intermediate Provenance Size.
Query Q_{TopK} (shown below) is used for T7: Interm. Prov. Size.

```
SELECT gb, min(va) AS mva, min(vb) AS mvb
FROM R^10M_fp
GROUP BY gc
ORDER BY mva
LIMIT < k
```

For Q_{TopK}, the total size of provenance is ∼ 10M · P_{SF}. We vary the number of tuples returned by `LIMIT` operator by controlling the paramter $k \in \{1K, 10K, 100K, 500K\}$ to control $P_{surviving}$:

$$P_{surviving} \in \{\frac{1}{2 \cdot P_{SF}}, \frac{1}{10 \cdot P_{SF}}, \frac{1}{100 \cdot P_{SF}}, \frac{1}{1K \cdot P_{SF}}\}$$

δ	Join Condition	#P $(\sum_i r_i) \cdot (\sum_j s_j))$	#SOP(P) $(\sum r_i \times s_i)$	P_{fact}
da	R.da = S.da	$2 \cdot 10K \cdot P_{SF}$	$2 \cdot 10K \cdot P_{SF} \cdot 10K \cdot P_{SF}$	$\frac{1}{10K \cdot P_{SF}}$
	R.da = S.db	$(10K + 200) \cdot P_{SF}$	$2 \cdot 10K \cdot P_{SF} \cdot 200 \cdot P_{SF}$	$\sim \frac{1}{400 \cdot P_{SF}}$
	R.da = S.dc	$(10K + 10) \cdot P_{SF}$	$2 \cdot 10K \cdot P_{SF} \cdot 10 \cdot P_{SF}$	$\sim \frac{1}{20 \cdot P_{SF}}$
db	R.db = S.db	$2 \cdot 200 \cdot P_{SF}$	$2 \cdot 200 \cdot P_{SF} \cdot 200 P_{SF}$	$\frac{1}{200 \cdot P_{SF}}$
	R.db = S.dc	$(200 + 10) \cdot P_{SF}$	$2 \cdot 100 \cdot P_{SF} \cdot 10 \cdot P_{SF}$	$\sim \frac{1}{20 \cdot P_{SF}}$
dc	R.dc = S.dc	$(10 + 10) \cdot P_{SF}$	$2 \cdot 10 \cdot P_{SF} \cdot 10 \cdot P_{SF}$	$\frac{1}{10 \cdot P_{SF}}$

Figure 11: `DISTINCT` P_{fact}

A.2.4 Provenance Factorization. Q_{JD} (shown below) used in T9: Factorization is a `DISTINCT` on top of a join. As mentioned in **??**, we control P_{fact} (how factorizable the provenance is) by select the result attribute A and the attribute from S we join on (B). Note that both queries produce the same final result and provenance and just differ in the factorization they induce (for systems that support that). Query Q_{DJ} (shown below) additionally eliminates duplicates before joining R and S:

Q_{JD}:
```
SELECT DISTINCT R.A
FROM dj1 AS R JOIN dj2 AS S
ON (R.A = S.B)
```

Q_{DJ}:
```
SELECT DISTINCT R.A
FROM (SELECT DISTINCT A FROM dj1) AS R
  JOIN (SELECT DISTINCT A FROM dj2) AS S
  ON R.A = S.B;
```

Note that the two joined tables have the same schema and data distribution. Each table has $10K \cdot P_{SF}$ rows. Both A and B are chosen from one of three attributes: *da* (one distinct value), *db* (50 distinct values) and *dc* (1*k* distinct values). We evaluate each pair of these three attributes from the two tables. For the first query Q_{JD} which yields a polynomial of the form $\delta(\sum r_i \times s_i)$, P_{fact} is 1 independent of the choice of A and B as the provenance for every result tuple is already expressed as a flat sum-of-products. Fig. 11 shows P_{fact} for the second query.

Additionally, in this task we use Q_{JA}, an aggregation on top of a join.

```
SELECT A_grp, avg(va) AS ava, avg(vb) AS avb
FROM R^10M_fp AS R JOIN R^c_join AS S ON A_grp = c1to10
GROUP BY A_grp
```

The factorized version, Q_{AJ}, pushes the aggregation into the left side of the join, yielding factorized provenance:

```
SELECT A_grp, sum(sva) / sum(cntva) as ava,
       sum(svb) / sum(cntvb) as avb
FROM (
  SELECT A_grp, sum(va) AS sva, sum(vb) AS svb,
         count(va) as cntva, count(vb) as cntvb
  FROM R^10M_fp GROUP BY A_grp
) AS R JOIN R^c_join AS S ON A_grp = c1to10
GROUP BY A_grp
```

For these two query template, we vary the parameter number of groups in the output of group-by $n \in \{10 \cdot P_{SF}, 1K \cdot P_{SF}\}$ controlled by varying the group by attribute A_{grp}. Query Q_{JA} produces a flat sum-of-products $\delta(\sum r_i \cdot s_i)$ and Q_{AJ} returns factorized polynomials of the form $\delta(\sum((\sum_i r_i) \cdot s_i))$. The number of groups is $1K * \cdot P_{SF}$ and each group has $100K \cdot P_{SF}$ tuples. For the first aggregation query we have $P_{fact} = 1$. For the factorized query, $P_{fact} \sim \frac{1}{10}$.

A.3 Query Constructs & Structure

A.3.1 Subquery. We use the following query template Q_{WSub} varying the number of subqueries in the `WHERE`. The template access two tables: $R^{100}_{\#\text{grp}}$ and R^{c}_{join}. We fix the result size and vary the total provenance size ($P_{\text{tot}} \in \{10^6 \cdot P_{\text{SF}}, 10^7 \cdot P_{\text{SF}}, 10^8 \cdot P_{\text{SF}}\}$) by increasing $10X$ for one additional subquery introduced, varying the average provenance size per result tuple ($P_{\text{res}} \in \{10^4 \cdot P_{\text{SF}}, 10^5 \cdot P_{\text{SF}}, 10^6 \cdot P_{\text{SF}}\}$)

```
SELECT ga, avg(va)
FROM R^100_#grp v
WHERE
  EXISTS (SELECT 1 FROM R^c_join j1 WHERE j1.c1to10 = v.ga)
  [
   AND EXISTS (SELECT 1 from R^c_join j2 where j2.c1to10 = v.ga)
   AND EXISTS (SELECT 1 from R^c_join j3 where j3.c1to10 = v.ga)
  ]
GROUP BY ga
```

Table 6 presents the query templates discussed in this section. Please see Sec. E for details of join format.

B ADDITIONAL SYSTEM DETAILS

We now present additional details on evaluated systems including discussing how these systems capture provenance for an example query and how benchmark tasks are implemented for each system.

B.1 Provenance System Evaluation Protocol

We now discuss in detail what versions of the systems we used in the benchmark and how tasks that go beyond mere capture are implemented in each system.

B.1.1 GProM. GProM is available on github: https://github.com/IITDBGroup/gprom. For PBench, we use git commit `06a1ea4b`. GProM allows the user to control what information to propagate as provenance. The default is to propagate all attributes of input tables.

Provenance Capture. For tasks where the goal is to just capture which input row identifiers contribute, we did only propagate row identifier attributes (e.g., single attribute keys or system row identifiers). This done by adding `USE PROVENANCE(x)` to a `FROM` clause item to instruct the system to use column x as provenance (instead of all columns). If the workload just requires row identifiers in the provenance to be returned, we project away non-provenance attributes. For example consider two tables $R(\underline{a}, b)$ and $S(\underline{c}, d)$ where primary key attributes (used as row identifiers for provenance tracking) are underlined. For the following query,

```
SELECT sum(a) AS sa, b
FROM R, S
WHERE b = c
GROUP BY b;
```

this is the query submitted to GProM:

```
SELECT prov_r_a, prov_s_c -- if only provenance is requested
FROM (PROVENANCE OF (SELECT sum(a) AS sa, b
                     FROM R USE PROVENANCE (a),
                          S USE PROVENANCE (c)
                     WHERE b = c
                     GROUP BY b);
```

Forward and backward debugging. For forward and backward debugging, we add additional filter operations on top of provenance requests to fetch the desired tuples. For example, since GProM captures the provenance alongside with the result, backward debugging tasks can be done by filtering for erroneous outputs to fetch the corresponding provenance. For the example query above, to trace the provenance of a result tuple $(1500, x)$, we submit the following query to GProM:

```
WITH prov AS
(
PROVENANCE OF (SELECT sum(a) AS sa, b
                      FROM R , S
                      WHERE b = c
                      GROUP BY b
)
SELECT *
FROM
(SELECT DISTINCT prov_r_a ,prov_r_b FROM prov
 WHERE sa = 1500 AND b = 'x')
  JOIN
(SELECT DISTINCT prov_s_c , prov_s_d FROM prov
 WHERE sa = 1500 AND b = 'x')
WHERE prov_r_b = prov_s_c;
```

For forward debugging a subset of the results should be returned which have an erroneous row in their provenance. For example for a tuple $(145, x) \in R$, we would issue:

```
WITH prov AS
( PROVENANCE OF (SELECT sum(a) AS sa, b
                    FROM R , S
                    WHERE b = c
                    GROUP BY b
)
SELECT sa, b
FROM prov
WHERE prov_r_a = 145 AND prov_r_b = 'x';
```

Reproducibility. For reproducibility tasks, we propagate all attributes of provenance tuples, and retrieve and materialize the provenance data. For instance, for the example query from above:

```
CREATE TABLE materialized_prov AS
(
    SELECT *
    FROM (PROVENANCE OF (SELECT sum(a) AS sa, b
                         FROM R, S
                         WHERE b = c
                         GROUP BY b)
);
```

Then the provenance tuples from each table are materialized:

```
CREATE TABLE prov_r AS
(SELECT DISTINCT prov_r_a, prov_r_b FROM materialized_prov);

CREATE TABLE prov_s AS
(SELECT DISTINCT prov_s_c, prov_s_d FROM materialized_prov);
```

The query's result is then reproduced by querying this materialized provenance:

```
SELECT sum(a) AS sa, b
  prov_r, prov_s
  WHERE prov_r_b = prov_s_c
  GROUP BY prov_r_b;
```

B.1.2 ProvSQL. ProvSQL is available on github: https://github.com/PierreSenellart/provsql. The system is constructed as a Postgres extension. We build the extension based on git commit `c4a0afd5`.

To add variables as initial provenance to a table R in the database using `SELECT add_provenance('R')`. During provenance capture, the size of mmap files, storing the gates of all provenance circuits for tables in a database and for (intermediate) results of a queries accessing these table, grows rapidly. Once the files reach approximately ~ 200GB, the system is no longer able to handle queries reliably. To ensure a consistent experimental setup, we create a backup of the mmap'ed files after constructing provenance tokens for all required tables. Before each experiment, the current mmap files are discarded and restored from the backup. This guarantees that every query is executed from the same initial provenance state and prevents performance degradation and failures caused by exhausting the available disk space.

For workloads concerning provenance structure & size and query constructs & structure, we directly execute the benchmark queries and measure the performance. For any query accessing tables with provenance tokens (where `SELECT add_provenance('R')` was run for the tables), ProvSQL automatically rewrites the query to construct circuits for the provenance polynomials of result tuples and returns and additional column storing the root of a sub-circuit corresponding to the provenance polynomial for a result tuple.

For use-case specific tasks, during provenance capture, we transform polynomials using the `src_which()` function which outputs Lineage as arrays of inputs' identifiers rather than `uuid` values, which makes it efficient to retrieve the original tuples when needed. For reproducibililty tasks, we re-construct query results by re-executing queries over tuples identified from the captured provenance. Let us reuse the tables `R` and `S` and query defined in Sec. B.1.1. The three queries below show the breakdown steps (capture, query provenance and reproducibility) to complete a reproducibility task. Note that we show the example of query provenance of table `R`.

```
CAPTURE:
CREATE TABLE materialized_prov_provsql AS
(
SELECT UNNEST(provone::integer[]) AS provone,
  UNNEST(provtwo::integer{}) AS provtwl
FROM (
  SELECT sr_which(provenance(), 'mapping_r') AS provone,
         sr_which(provenance(), 'mapping_s') AS provtwo

  FROM (SELECT sum(a) as sa, b
        FROM R, S WHERE b = c
        GROUP BY b)
);

QUERY:
CREATE TABLE prov_r
SELECT R.a, R.b
FROM (
  SELECT DISTINCT provone
  FROM materialized_prov_provsql
), R
WHERE R.a = prov_r.provone;

CREATE TABLE prov_s
SELECT S.a, S.b
FROM (
  SELECT DISTINCT provone
  FROM materialized_prov_provsql
), S
WHERE S.c = prov_r.provtwo;
```

```
REPRODUCIBILITY:
SELECT sum(a) AS sa, b
FROM prov_r, prov_s
WHERE b = c
GROUP BY b;
```

For backward debugging tasks, it differs from reproducibility task in the last step, in which we fetch the provenance for erroneous result tuples (according to `GROUP BY` value) shown below:

```
SELECT R.a, R.b, S.c, S.d
FROM prov_r, prov_s
WHERE b = c
  AND R.b = 'x'
;
```

For forward debugging, we query the result data with the corresponding provenance (shown below).

```
CREATE TABLE materialized_prov_provsql AS
(
SELECT sa as sa, b as b, provone AS provone,
   provtwo AS provtwo
FROM (
  SELECT sr_which(provenance(), 'mapping_r') AS provone,
         sr_which(provenance(), 'mapping_s') AS provtwo

  FROM (SELECT sum(a) as sa, b
        FROM R, S WHERE b = c
        GROUP BY b)
);
```

Since the aggregation result of ProvSQL cannot be used for further computation, we fetch the computable version of aggregation result by joining the result with materialized table (shown below).

```
CREATE TABLE materialized_prov_res AS
(
  SELECT U.sa as sa, T.b as b,
    T.provone as provone, T.provtwo as provtwo
  FROM (
    SELECT sum(a) AS sa, b
    FROM R, S WHERE b = c
    GROUP BY b
  ) U, materialized_prov_provsql T
  WHERE T.b = U.b
)
```

Then we fetch the impacted result rows by checking whether erroneous inputs contribute to results.

```
SELECT sa, b
FROM materialized_prov_res
WHERE EXISTS (
  SELECT 1
  FROM R
  WHERE R.a = 145
     AND R.b = 'x'
     AND R.a = ANY(materialized_prov_res.provone::integer[])
);
```

B.1.3 SQLProv. The full code of SQLProv is not publicly available. We reached out to the authors and were provided with code and instructions that enabled us to run their approach. The SQLProv follows the same execution protocol as ProvSQL, as both systems capture provenance using tuple' identifiers. SQLProv materializes data into logging tables during the first phases. For example, reusing our running example from Sec. B.1.1 of the aggregation-join query, SQLProv logs the joining result of `R` and `S` and the aggregation result into materialized tables `logjoin` and `logaggregation` storing each

tuple's `tuid`. Before execution of each query, we truncate the corresponding log tables, ensuring accurate measurements of storage overhead.

B.1.4 SmokedDuck. SmokedDuck is available on github (https://github.com/cudbg/sd) And over pipy (https://pypi.org/project/smokedduck/). However, the version on pipy crashes with a segfault. We were in contact with the authors and based on their feedback did use the code available in the branch https://github.com/cudbg/sd/tree/smokedduck-2025-f using git commit `5c0e89d4`. The different branches of SmokedDuck differ in how backtracing in phase II operates. Some versions provide an interface where the user provides a single output `rowid` and the system traces back provenance for that row only using indexes that are build in memory for the captured provenance after phase I while others construct a query that joins the mapping tables between output and input rowids of operators written during phase I to return the full provenance. The branch we utilized was using the full rowids backtracking. While we were able to run phase I successfully with this version of SmokedDuck, we encountered issues with phase II. With feedback from the authors and our own investigation we were able to get most of the queries running.

SmokedDuck is a forked version of duckdb based on duckdb version `1.0.0`. Furthermore, the system was only functional when limiting parallel query execution to a single thread (`SET threads=1;`). For fairness of comparison we used this version in the evaluation for GProM too. However, as mentioned in Sec. 7.4 this penalizes GProM as newer versions of DuckDB are significantly better in handling the more complex rewritten queries produced by GProM for provenance capture.

For use-case tasks, we use the version (https://github.com/cudbg/sd/tree/lineage_capture_v5) with the commit `3569be24` that can trace back full provenance in phase II. For reprodicibility and forward debugging tasks, SmokedDuck follows the same task protocol as ProvSQL and SQLProv. Similar to these systems, SmokedDuck represents provenance using row identifiers, although it relies on table `rowids` instead of defined tuple identifiers. For forward debugging, the protocol is mostly the same. However, since SmokedDuck only returns `rowids` of provenance tuples, in order to complete the tasks, we first associate each result tuple with it provenance using `out_index` field in the retrieved provenance. We then check the affected result rows by identifying whether their provenance contains erroneous input rows. Suppose we materialize provenance traced from phase II into a table, `prov_smokedduck`, and the result of query into another materialized table, `res_smokedduck`, we use the following query to fetch the affected rows to complete the task.

```
SELECT U.sa, U.b
FROM prov_smokedduck AS T,
     res_smokedduck AS U,
     R
WHERE T.out_index = U.rowid -- mapping result with provenance
  AND R.rowid = T.r
  AND R.a = 145 and R.b = 'x';
```

B.2 Example Provenance Capture Process

Fig. 12 shows the capture process for query Q_{emp} from Fig. 1 that we repeat here for convenience:

```
SELECT name, salary
FROM  Dept JOIN Emp ON (did = deptid)
WHERE region = 'North';
```

GProM. In Fig. 12, attributes `Emp.cid` and `Dep.did` are selected as provenance for input rows (using the `USE PROVENANCE(x)` construct to select for each base table the attributes x to track. The query send to GProM is:

```
PROVENANCE OF (
    SELECT name, salary
    FROM  Dept USE PROVENANCE (did)
          JOIN
          Emp USE PROVENANCE (cid)
          ON (did = deptid)
    WHERE region = 'North';
);
```

GProM instruments the query to propagate the provenance attributes, result in the following query:

```
SELECT F1_0.name AS name,
       F1_0.salary AS salary,
       F0_0.prov_dept_did AS prov_dept_did,
       F1_0.prov_emp_eid AS prov_emp_eid
FROM (SELECT F0_0.did AS did,
             F0_0.did AS prov_dept_did -- duplicate
                                       -- provenance attr
      FROM (SELECT F0_0.did AS did,
                   F0_0.region AS region
            FROM dept F0_0) F0_0
      WHERE (F0_0.region = 'North')) F0_0
      JOIN (SELECT F0_0.name AS name,
                   F0_0.deptid AS deptid,
                   F0_0.salary AS salary,
                   F0_0.eid AS prov_emp_eid  -- duplicate
                                             -- provenance
                                             --  whattr
            FROM emp F0_0) F1_0
      ON ((F0_0.did = F1_0.deptid)))
```

In the result of the instrumented query each tuple is paired with a `cid` and `did` value that encodes which input tuples were joined to derive this result. For example, (*Alice*, 110000) is the result of joining the `Emp` tuple with `cid=1` and department tuple with `did=101`. **ProvSQL.** To run the query with provenance tracking, it is required that previously the input tables accessed by the query were extended to store a variable associated with the row, e.g., for table `Emp` by running `SELECT add_provenance('Emp');`. ProvSQL automatically detects that a query is accessing a provenance enabled table and instruments the query to create circuit gates for provenance polynomials of intermediate results and the final query result. The invariant maintained is that every (intermediate) result tuple is extended with an extra column storing the UUID of the root of a circuit that encodes the provenance polynomials for each tuple. Note that subcircuits are shared. query shown in Fig. 12,

the result tuple for Alice is associated with a gate with UUID u_6 which is a +-gate that is the root of the circuit encoding the polynomial $t_1 \cdot s_1$ for this result.

SmokedDuck. In the first phase – shown on the left of SmokedDuck in Fig. 12 – the system executes query and materializes the indices (`rowids` of a table) of tuples into tables (show in yellow color on the right).

Within the materialized index tables, `out_index` starts from 0 and increments by 1 for each subsequent tuple. The `out_index`

in the materialized table of a operator serves as the `in_index` in the materialized table of its parent operator except for table access operator where `in_index` is the `rowid` of a tuple. Similarly, `lhs_index` (`rhs_index`) in materialized table of a join operator works as `in_index`.

In the second phase (show on the right), the system joins the materialized index tables to back trace the provenance. It outputs the provenance using `rowids` of the contributing source tuples. For example, provenance for (*Alice*, 110000) is $\{0, 0\}$ (assuming the `rowid` of two joined tuples are 0 and 0).

SQLProv. SQLProv executes the query using the tables built for this phase (in our example `Emp1 and Dept1`), and logs the `tuid` (unique identifier for a tuple) of joined tuples into table `LogJoin` in the first phase. In the second phase, the system bottom-up computes provenance joining original tuples' `tuids` from tables of `Emp2` and `Dept2` (built for the second phase) with the materialized `tuids` and outputs `tuids` of contributed tuples. For example, the provenance of result tuple (*Alice*, 110000) is $\{20, 1\}$. Note that the `tuids` for a tuple in constructed tables used by the first and second phases are the same.

C ADDITIONAL BENCHMARK TASKS

C.1 Use-case Specific Tasks

C.1.1 Forward Debugging. In **T13: Forward debug - impacted queries** the system is provided with a set of queries and a set of rows from the database that are considered to be erroneous. This task is then to determine for each query Q in the set whether it has at least one of the erroneous rows in its provenance, i.e., the query's result is potentially affected by the error. We use a query template, Q_{FDQ} (shown below),

```
SELECT ga, avg(va) AS ava, avg(vb) AS avb
FROM R^{1k}_{#grp}, R^{c}_{join}
WHERE ga = c1to10 and ga >= 1 and ga <= 333
GROUP BY ga
```

working on two synthetic tables: $R^{1k}_{\#grp}$ and R^{c}_{join}. We vary the error data rage in table $R^{1k}_{\#grp}$ $P_{\%|D|}$ in 1%, 10% and 30% of total provenance size $1M \cdot P_{SF}$. The last task **T14: Forward debugging - provenance and result** maps the erroneous provenance tuples with the affected result tuples, and utilizes the same data and query settings as T2.

C.1.2 Reprodicibility. **T15: Reeval. - coarse-grained** uses coarse-grained provenance. For this task we consider the scenario of using provenance captured for one query to answer another future query [48, 61]. In this context, coarse-grained provenance has the advantage of small storage size. We use a query Q_{AP} (shown below), and vary the number of fragments, rows of table R^{10M}_{fp}, $10M \cdot P_{SF}$, are grouped into, which determines the granularity of the captured provenance $P_{\#frag} \in \{10, 100, 500\}$.

```
SELECT gb, avg(va) AS ava
FROM R^{10M}_{fp} JOIN R^{c}_{join} on gb = c1to10
GROUP BY gb HAVING avg(va) < 15
```

As in the reproducibility tasks, the coarse-grained provenance has to be captured and then in a separate step is used to answer a query.

C.2 Query Constructs & Structure Tasks

In this part, we provide additional design of varying structure of queries and what query constructs are used in PBench.

C.2.1 Complex Join Conditions. **T16: Complex Join** uses a join query with a more complex join condition expressed as a disjunctive `WHERE` condition followed by an aggregation. These complex conditions prevent database engines from exploiting the standard optimizations , such as selection-push-down. Thus, this creates a computationally intensive environment that forces the systems to fully evaluate the join predicates during the capture process. We use a query Q_{CplxJ} (shown below) which joins tables $R^{1k}_{\#grp}$ and R^{c}_{join} and each tuple from $R^{1k}_{\#grp}$ has 10 join partners. We control the number of conjunctive predict blocks, $n \in \{1, 2, 3\}$, that can form a disjunction. The intermediate result sizes are $5M \cdot P_{SF}$, $10M \cdot P_{SF}$ and $15M \cdot P_{SF}$ for n ranging from 1 up to 3.

```
SELECT ga, count(vb) AS cvb, avg(vb) AS avb
FROM R^{1k}_{#grp}, R^{c}_{join}
WHERE
     (ga >= 0 AND ga <= 300
      AND va >= 20 AND va <= 280
      AND vb >= 150 and vb <= 250
      AND ga = c1to50)
     [
       OR (ga >= 301 AND ga <= 600
           AND va >= 320 AND va <= 580
           AND vb >= 450 AND vb <= 550
           AND ga = c1to50)
       OR (ga >= 601 AND ga <= 900
         AND va >= 620 AND va <= 880
         AND vb >= 750 AND vb <= 850
         AND ga = c1to50)
     ]
GROUP BY ga
```

C.2.2 Multi-Level Aggregation. **T17: Multi-level Aggr.** applies a multi-level aggregation query. Such queries challenge provenance systems like GProM that compute provenance bottom-up through propagation as it significantly increases the size of intermediate results. We use query Q_{MLAgg} (shown below) over a single table R^{10M}_{fp}, $10M \cdot P_{SF}$. We fix the total provenance size and control the number of groups $n \in \{1K \cdot P_{SF}, 1M \cdot P_{SF}\}$ in the inner aggregation by varying the group by attribute A_{grp}. Q_{MLAgg} first computes n aggregation tuples which are then further aggregated into a single row by the outer aggregation.

```
SELECT max(va) AS maxava, max(vb) AS maxvb
FROM (SELECT gb, avg(va) AS va, avg(vb) AS vb
      FROM R^{10M}_{fp} GROUP BY A_{grp}) AS sub
```

C.2.3 Argmin. **T18: Argmin** is the argmin query Q_{AMin} (shown below) which is a one-to-ten join query that computes the minimum for a group and then joins the aggregation result back to the input table to retrieve additional attributes from the row(s) with the minimum value. This query runs over two tables, $R^{n}_{\#grp}$ and R^{c}_{join}, and yields a factorized polynomial $(\sum_i r_i) \cdot s_i$. We vary the parameter $n \in \{100, 1K\}$ to vary the total provenance size in $\{\sim 100K \cdot P_{SF}, \sim 1M \cdot P_{SF}\}$ but fix the per tuple provenance size, $1K \cdot P_{SF}$.

```
SELECT ga, minv, c1to10
FROM (SELECT ga, min(va) AS minv
      FROM R^{n}_{#grp}
```

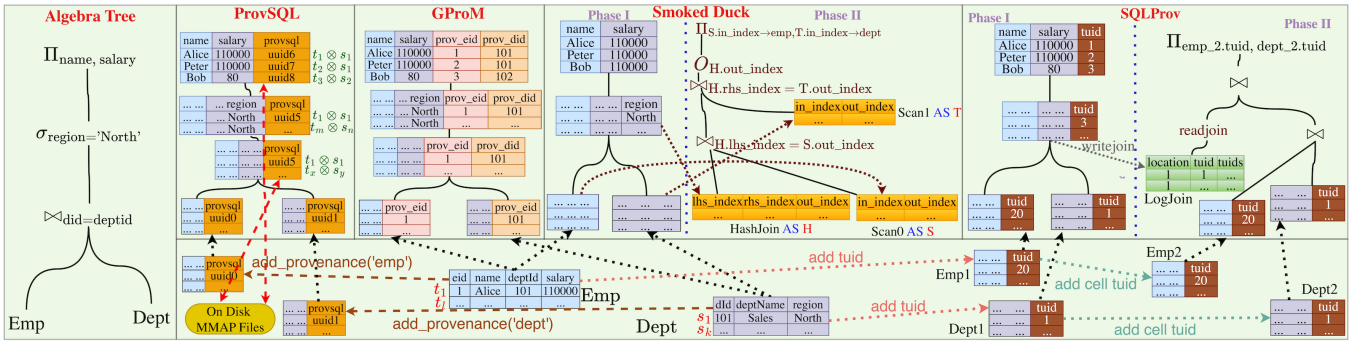


Figure 12: Comparing provenance capture in different systems for example query Q_{emp}

```
    GROUP BY ga) AS R
JOIN R^c_join AS S ON R.minv = S.c1to10
```

C.2.4 Recursive. **T19: Recursive Query** uses a recursive query Q_{RCR} (shown below) that returns the end points and lengths of paths in a graph (relation R_{graph}) that have less than d steps. We vary parameter $d \in \{2, 3\}$. The R_{graph} represents a dense random graph where each node has an average out degree of 100. Thus, Increasing the d will lead to an exponential expansion of search space. The size of the provenance for this query grows significantly in d.

```
WITH RECURSIVE rcr AS (
    SELECT f, t, 1 AS d
    FROM R_graph
    WHERE f <=10 · P_SF
    UNION ALL
    SELECT p.f AS f, p.t AS t, r.d + 1 AS d
    FROM rcr as r , R_graph as p
    WHERE r.t = p.f and r.d < d
)
SELECT * FROM rcr;
```

C.2.5 Window Function. **T20: Window Func.** evaluates provenance capture for queries containing window functions. Window functions can result in large provenance. For instance, when using windows with `UNBOUNDED PRECEDING` the provenance of a row contains all rows that precede this row in sort order. However, such provenance is highly compressible as the provenance of consecutive rows overlap to a large degree. We us three variants of query Q_{Win} (shown below) accessing table R^{10M}_{fp}, size of $10M \cdot P_{SF}$: the first two queries use `PARTITION BY`, varying group by attribute A_{grp} to vary the number of groups in $\{10 \cdot P_{SF}, 1k \cdot P_{SF}\}$, resulting in provenance of limited size per result row while the third omits the `PARTITION BY` clause, resulting in large provenance (on average half the input table is in the provenance of a result tuple).

```
SELECT ga, avg(va) OVER ([PARTITION BY A_grp]
                         ORDER BY id) AS rkava
FROM R^10M_fp;
```

C.2.6 Limit Operator. **T21: Limit Op.** evaluates provenance capture for queries having limit operator (non Top-k). Limit operator can examine if a provenance system exploits the optimization to short-circuit execution and terminate early at the moment it hits the limit boundary. And limit operator test a system with large intermediate result but small final provenance size of query result. We use the query template Q_{Lmt} (shown below), applying `LIMIT` operation on top of group-by-aggregation, accessing a single table R^{10M}_{fp}. The total size of provenance is fix and the total number of tuples inputted into `LIMIT` is $P_{SF} \cdot 1M$. We vary the $P_{surviving}$, $P_{surviving} \in \{0.1\%, 1\%, 10\%\}$ by controlling the number of tuples, $n \in \{1K, 10K, 100K\}$, `LIMIT` restricts.

```
SELECT gc, avg(va) AS ava, avg(vc) AS avc
FROM R^10M_fp
GROUP BY gc
LIMIT n
```

C.2.7 Set Operator. **T22: Set Op.** evaluates provenance capture for negative queries using set operator `EXCEPT`. We use the query templateQ_{Set} (shown below) accessing a single table R^{10M}_{fp}. The total provenance size is fixed, $10M \cdot P_{SF}$ and the amount of data from both sides of `EXCEPT` is the same. We vary the data remove rate, r, of left side to right side of `EXCEPT` operator. For example, $r = 0.1$ indicates that 10% data from both sides are the same. The remove rate is controlled by varying value range, rng, in right side.

```
(SELECT gb FROM fp WHERE gb <= 500)
EXCEPT
(SELECT gb FROM fp WHERE rng)
```

Table 7 presents an overview of query templates discussed in Sec. C. Please refer Sec. E for the details of join format.

D ADDITIONAL EXPERIMENTAL RESULTS

D.1 Use-case Specific Tasks

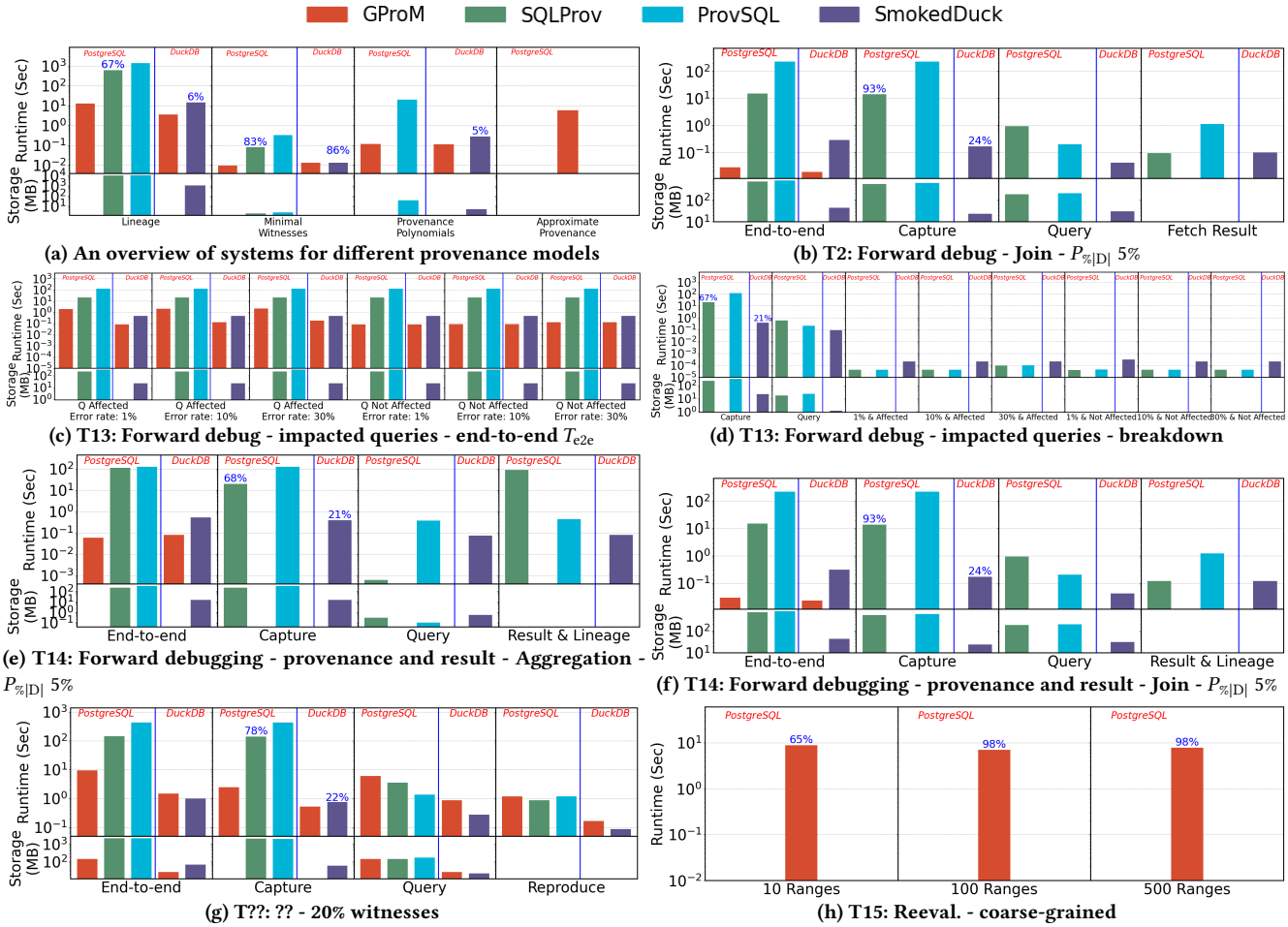
T2: Forward debug. Fig. 13b and show the end-to-end and breakdowns evaluation for template and Q_{FD2}. The result show that the capture cost dominate the over all performance of ProvSQL, SmokedDuck and SQLProv.

T13: Forward debug - impacted queries. Fig. 13c and Fig. 13d show show the end-to-end and breakdowns evaluation for templates Q_{FDQ}. GProM shows the best performance as there is possible to add additional computation on top of query result to filter the desired result. While multiple steps are required for SmokedDuck, SQLProv and ProvSQL since the provenance has to be retrieved to further process to detect which queries are affected by the erroneous input data.

T14: Forward debugging - provenance and result. Fig. 13e and Fig. 13f present the evaluation for this use case Q_{FD1} and Q_{FD2}.

Task	Query	Tables	Parameter	Join Format	Tot. Prov.	Inter. Res.	Res.
T13: Forward debug - impacted queries	Q_{FD1}	$R^{1K}_{\#grp}, R^{c}_{join}$	–	**1000%(1 − 10)**	~ 1M	10M	1K
T14: Forward debugging - provenance and result	Q_{FD1}	$R^{1K}_{\#grp}, R^{c}_{join}$	–	**1000%(1 − 10)**	~ 0.33M	10M	333
	Q_{FD2}	$R^{1K}_{\#grp}, R^{c}_{join}$	–	**1000%(1 − 10)**	~ 0.33M	10M	~ 3.3M
T15: Reeval. - coarse-grained	Q_{AP}	$R^{10M}_{fp}, R^{c}_{join}$	–	**1000%(1 − 10)**	~ 10M	100M	15
T16: Complex Join	Q_{CplxJ}	$R^{1K}_{\#grp}, R^{c}_{join}$	$n \in \{1, 2, 3\}$	**500%(1[10%] − 50), 1000%(1[20%] − 50), 1500%(1[30%] − 50)**	~ 0.1M, ~ 0.2M, ~ 0.3M	~ 5M, ~ 10M, ~ 15M	{100, 200, 300}
T17: Multi-level Aggr	Q_{MLAgg}	R^{10M}_{fp}	$n \in \{1K, 1M\}$	–	~ 10M, ~ 15M	10M	{1K, 1M}
T18: Argmin	Q_{AMin}	$R^{c}_{\#grp}, R^{c}_{join}$	$n \in \{100, 1K\}$	**1000%(1 − 10)**	~ 0.1M, ~ 1M	{1M, 10M}	{1K, 10K}
T19: Recursive Query	Q_{RCR}	R_{graph}	$d \in \{2, 3\}$	–	1 K	–	~ 110K, ~ 1.1M
T20: Window Func	Q_{Win}	R^{10M}_{fp}	w/o partition, w/ partition(A_{grp})	–	10M	–	10M
T21: Limit Op	Q_{Lmt}	R^{10M}_{fp}	$n \in \{1K, 10K, 100K\}$	–	10M	1M	{1K, 10K, 100K}
T22: Set Op	Q_{Set}	R^{10M}_{fp}	$rng \in \{[401, 500], 250, 500], [1, 500]\}$	–	10M	–	{400, 250, 0}

Table 7: Query templates ($P_{SF} = 1$) discussed in Sec. C: tables accessed, varied parameters, join format, total provenance size (Tot. Prov.), intermediate result size (Inter. Res.) and result size (Res.)



(a) An overview of systems for different provenance models

(b) T2: Forward debug - Join - $P_{\%|D|}$ 5%

(c) T13: Forward debug - impacted queries - end-to-end T_{e2e}

(d) T13: Forward debug - impacted queries - breakdown

(e) T14: Forward debugging - provenance and result - Aggregation - $P_{\%|D|}$ 5%

(f) T14: Forward debugging - provenance and result - Join - $P_{\%|D|}$ 5%

(g) T??: ?? - 20% witnesses

(h) T15: Reeval. - coarse-grained

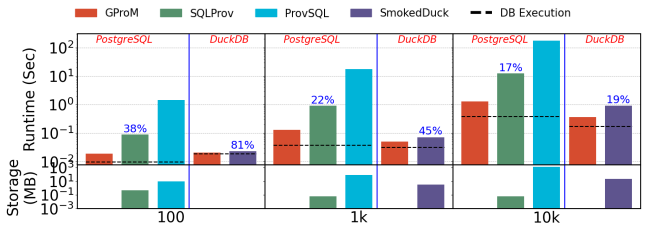
Figure 13: Use-case specific tasks: additional experiments

For glsgprom, we propagate all attributes of contributed tuples, making it easy to complete this task by retrieving the result and provenance data. Other systems follow our three-step protocol to finish this task. This task differs from T2: Forward debug in the last step where instead of fetching only affected result tuples, we retrieve the erroneous inputs as well as the affected result tuples.

T??: ??. Fig. 13g show the experimental result of query Q_{Lin} for reproducibility using provenance models lineage, minimal witness and 20% lineage. These figures shows (1) the end-to-end performance of all systems, and (2) performance breakdowns of systems SQLProv, ProvSQL and SmokedDuck. ProvSQL needs to read from and write to memory mapped files, and thus this leads worst performance among all systems. The minimal witness model requires least cost of time and storage to complete the reproducbility tasks, because the amount of total provenance data becomes much less compared to lineage model which has the worst performance. For SmokedDuck and SQLProv, the percentage of phase I over total

capture time increases when size of lineage decreases, because (1) phase II is more sensitive to the size of materialized data, and less lineage means less computation in phase II, and (2) phase I is a must-have phase even if a query produce no results. In terms of storage usage, lineage model is obviously largest. Note that for minimal witness, SmokedDuck consumes storage but data size is less than one block, and thus, actual value cannot be queried. Minimal witness model is an optimized for reproducing the result (reproduce cost) in terms of both execution and storage cost.

T15: Reeval. - coarse-grained. Fig. 13h presents the GProM's capability to capture the coarse-grained provenance. Compared to the table *fp* partitioned 10 ranges, number of ranges of 100 and 500 lead to higher cost in capture the provenance but less cost in reuse the sketches. Thus, the finer-grained partitioning leads to skip more irrelevant data to achieve faster query processing.



(a) Vary provenance size by varying size of groups

Figure 14: Provenance structure & size: additional experiments

D.2 Provenance Structure & Size

T6: Ttl. Prov. Size. Fig. 14a shows the runtime when varying the total provenance size P_{tot} by varying the size of group in an aggregation query. All systems show a trend of linear scale with total provenance size, indicating that total provenance dominates the cost of capture cost.

D.3 Query Constructs & Structure

T16: Complex Join. Fig. 15a shows the evaluation of system to capture provenance for the query Q_{CplxJ}. All systems show a trend of increasing runtime when the join condition blocks increase. Since the join predicates prevent the systems from applying query optimizations, they must evaluate a large number of join conditions during query execution. As a result, provenance capture becomes increasing expensive when the number of join predicates grows.

T17: Multi-level Aggr. Fig. 15b shows the experimental evaluation for Q_{MLAgg}. All systems show a trend of runtime increasing when there are more groups inside the inner aggregation, as it increases the total provenance size and the number of groups to be evaluated in the outer aggregation.

T18: Argmin. Fig. 15c presents the experimental evaluation of provenance capture for query Q_{AMin}. In this experiment, we fix the per result tuple provenance size P_{res} at 1K and vary the number of aggregation groups. Since the Q_{AMin} computes an aggregation first and then joins the aggregation result with join another table, increasing the number of groups directly increases the number of aggregation results, and thus, the number of tuples participating in join, leading to the increasing of total provenance size. GProM performs the worst as it cannot exploit the factorized provenance for queries.

T19: Recursive Query. Fig. 15d shows the provenance capturing performance for Q_{RCR}. The results show that the complexity of a path (i.e. the number of nodes in a path) determines the cost of performance. As the depth increases, provenance capture becomes more expensive because additional recursive iterations must be evaluated, which increase the provenance size and capture runtime.

T20: Window Func. Fig. 15e presents the evaluation for Q_{Win}. The results show that (1) introducing a `PARTITION BY` clause increases the provenance capture runtime, (2) the runtime further increases as the number of partitions grows.

T21: Limit Op. Fig. 15f shows the results of provenance capture for query template Q_{Lmt}. ProvSQL and SQLProv benefit when the `LIMIT` predict is selective, while GProM and SmokedDuck keep the performance stable when varying the number of tuples returned by `LIMIT`.

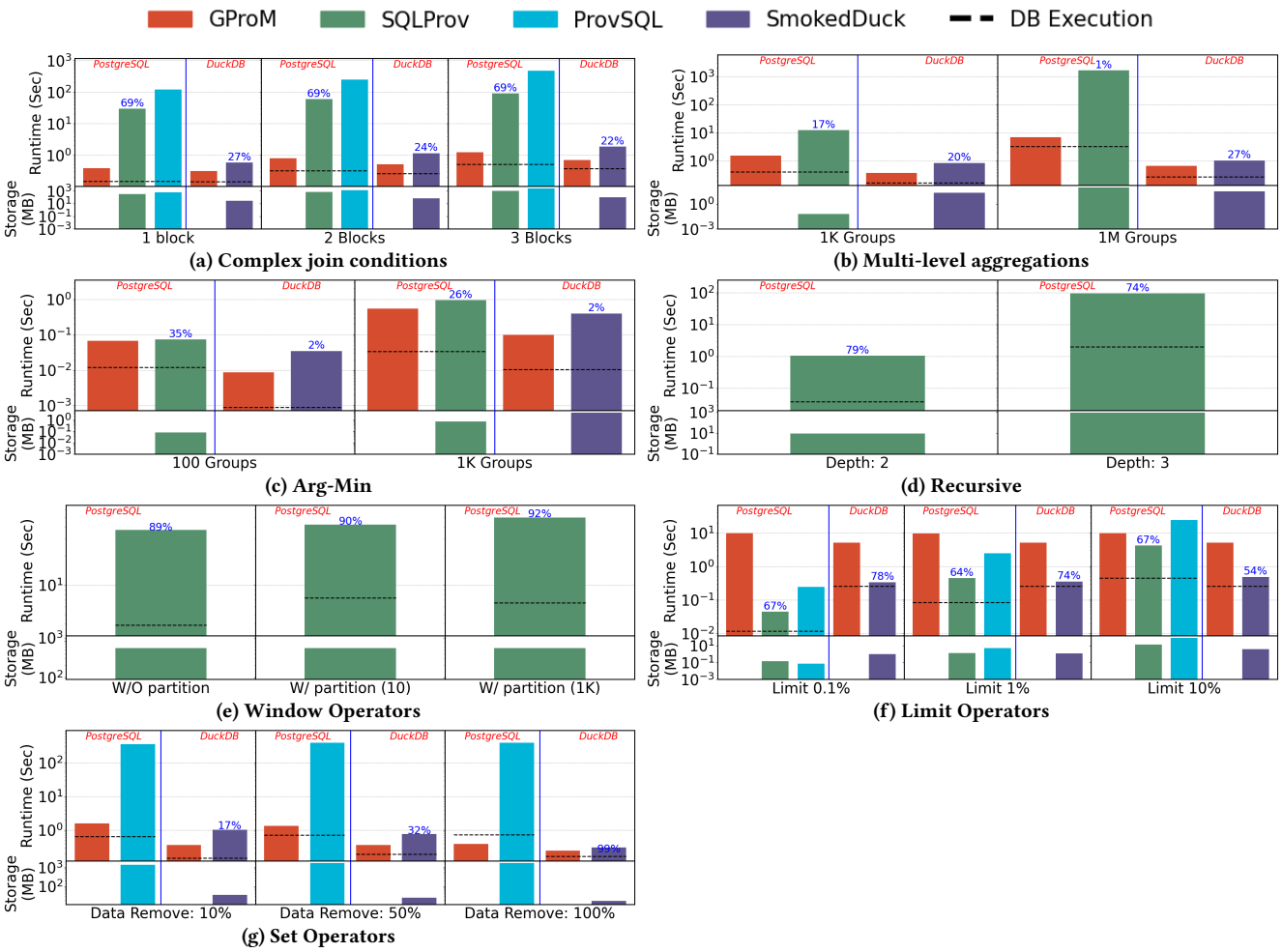
T22: Set Op. Fig. 15g present the result of evaluation system capture provenance for set difference, `EXCEPT`. SmokedDuck and GProM show a decreasing runtime trend as more data removed from left hand side of the operator, because less data remains for provenance capture. While the performance of ProvSQL keeps relatively constant, since for negative queries, it evaluates tuples that are potentially non-existence in the result result – a core mechanism designed to support incomplete databases.

E JOIN SELECTIVITY

We now discuss the selectivity of joins used in queries in the benchmark. Consider a star join query, where R_0 is the center of the star, i.e., R_0 joins with each R_i.

```
SELECT ...
FROM R0, R1, ..., Rn
WHERE ...
```

We use an unified notation for join query selectivity as *selectivity profiles* that compactly describe the selectivity in a query with multiple joins. The *selectivity profile* of such an n-way join is of the form $\boldsymbol{x(1[y] - j_1 - \cdots - j_n)}$. Here, x% is the join result size relative to the size of the left-most table R_0, e.g., $x = 2.0$ would imply that the result of the join is twice the size of relation R_0. Parameter y% is the fraction of tuples from the left-most table R_0 that have join partners with the result of joining all remaining tables. Furthermore, j_i is the average number of tuples from the $i + 1^{th}$ table that can join with a tuple from R_0. For example, $\mathbf{100\%(1[10\%] - 10)}$, is a query with one join whose output is the same size as R_0 due roughly 10% of tuples from R_0 having join partners, but each such tuple on average has 10 join partners in R_1. As another example, consider $\mathbf{500\%(1[10\%] - 10 - 5)}$ which describes a three-way join query which returns $5 \cdot |R_0|$ tuples, because on average every of the 10% tuples from R_0 that have join partners in both R_1 and R_2 do join with 10 tuples in R_1 and 5 tuples in R_2.



(a) Complex join conditions

(b) Multi-level aggregations

(c) Arg-Min

(d) Recursive

(e) Window Operators

(f) Limit Operators

(g) Set Operators

Figure 15: Query constructs & structure: additional experiments